

LAB-MANUAL

DSA

What is Data structure?

A data structure is a technique of storing and organizing the data in such a way that the data can be utilized in an efficient manner. In computer science, a data structure is designed in such a way that it can work with various algorithms.

The most important point in a data structure is:

- A large amount of data is organized through every type of data structure.
- A particular principle is followed by every data structure.
- The basic principle of the data structure should be followed even if any operations are carried out over the data structure.

Arrangement of the data within a data structure can follow different orders. A data structure is therefore classified according to the way of arrangement of the data. Basically, there are two **types of data structure**.

1. Primitive data structure
2. Non-primitive data structure

The **primitive** type of data structure includes the predefined data structures such as char, float, int, and double.

The **non-primitive** data structures are used to store the collection of elements. This data structure can be further categorized into

1. Linear data structure
2. Non-Linear data structure.

What is the Linear data structure?

Linear is a data structure in which the elements are stored sequentially, and the elements are connected to the previous and the next element. As the elements are stored sequentially, they can be traversed or accessed in a single run. The implementation of linear data structures is easier as the elements are sequentially organized in memory. The data elements in an array are traversed one after another and can access only one element at a time.

The types of linear data structures are Array, Queue, Stack, Linked List.

Let's discuss each Linear data structure in detail.

- **Array:** An array consists of data elements of the same data type. For example, if we want to store the roll numbers of 10 students, instead of creating 10 integer type variables, we will

create an array having size 10. Therefore, we can say that an array saves a lot of memory and reduces the length of the code.

- **Stack:** It is linear data structure that uses the LIFO (Last In-First Out) rule in which the data added last will be removed first. The addition of data element in a stack is known as a push operation, and the deletion of data element from the list is known as pop operation.
- **Queue:** It is a data structure that uses the FIFO rule (First In-First Out). In this rule, the element which is added first will be removed first. There are two terms used in the queue **front** end and **rear**. The insertion operation performed at the back end is known as enqueue, and the deletion operation performed at the front end is known as dequeue.
- **Linked List:** It is a collection of nodes that are made up of two parts, i.e., data element and reference to the next node in the sequence.

What is Searching?

Searching is the process of fetching a specific element in a collection of elements. The collection can be an array or a linked list. If you find the element in the list, the process is considered successful, and it returns the location of that element.

And in contrast, if you do not find the element, it deems the search unsuccessful.

Two prominent search strategies are extensively used to find a specific item on a list. However, the algorithm chosen is determined by the list's organization.

- Linear Search
- Binary Search

Moving ahead in this manual, you will understand what exactly binary search & linear search algorithm is.

Program 1:

Write a C++ Program for Binary Search.

Description:

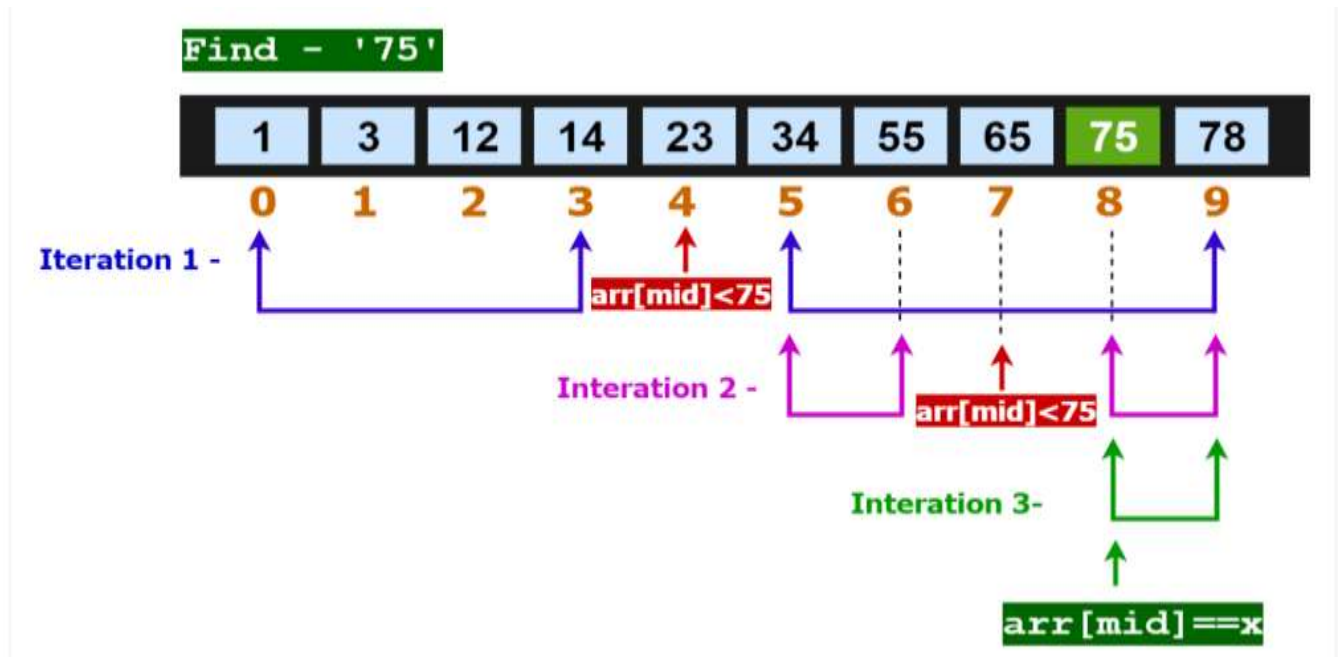
Binary search algorithm falls under the category of interval search algorithms. This algorithm is much more efficient compared to linear search algorithm. Binary search only works on sorted data structures. This algorithm repeatedly targets the center of the sorted data structure & divides the search space into half till the match is found.

The time complexity of binary search algorithm is **O (Log n)**.

Working:

1. Search the sorted array by repeatedly dividing the search interval in half.
2. Begin with an interval covering the whole array.
3. If the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half.
4. Otherwise narrow it to the upper half.
5. Repeatedly check until the value is found, or the interval is empty.

Binary Search Algorithm Diagram:



Algorithm to perform Binary Search:

1. Take input array, left, right & x
2. START LOOP - while(left greater than or equal to right)
 - $mid = left + (right - left) / 2$
 - if($arr[mid] == x$) then
 - return m
 - else if($arr[mid]$ less than x) then
 - $left = m + 1$
 - else
 - $right = mid - 1$
3. END LOOP
4. return -1

Implementation of Binary Search:

```
#include<iostream>
using namespace std;
int main()
```

```

{
    int myArray[6];
    int chk= -1;
    for(int i = 0 ; i < 6 ; i++)
    {
        cout<<"Index = "<<i<<endl;
        cin>>myArray[i];
    }
    int x ;
    cout<<"Which value do you wanna search:";
    cin>>x;
    int start = 0;
    int end = 5;
    int mid ;
    while(start<=end)
    {
        mid=(start+end)/2;
        if(myArray[mid]==x)
        {
            cout<<"Number is found on Index "<<mid<<endl;
            chk++;
            break;
        }
        else
        {
            if(x>myArray[mid])
            {
                start=mid+1;
            }
            else
            {
                end=mid-1;
            }
        }
    }
    if(start>end)
    {
        cout<<"Index not found "<<endl;
    }
    return 0;
}

```

```
C:\Users\Amir Asif\OneDrive\ >
Index = 0
4
Index = 1
8
Index = 2
9
Index = 3
7
Index = 4
94
Index = 5
100
Which value do you wanna search:100
Number is found on Index 5

-----
Process exited after 19.5 seconds with return value 0
Press any key to continue . . .

C:\Users\Amir Asif\OneDrive\ >
Index = 0
4
Index = 1
8
Index = 2
9
Index = 3
7
Index = 4
94
Index = 5
100
Which value do you wanna search:1
Index not found

-----
Process exited after 30.71 seconds with return value 0
Press any key to continue . . .
```

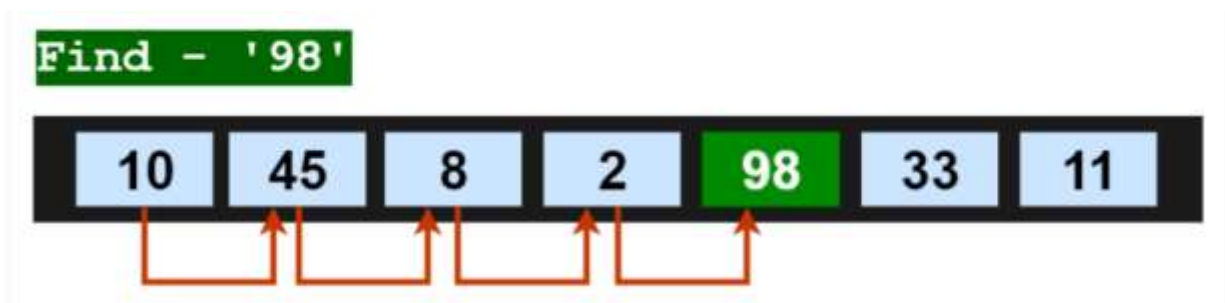
Program 2:

Write a C++ Program for Linear Search.

Description:

In computer science, a **linear search algorithm** or **sequential search** is a method for finding an element within a list. It sequentially checks each element of the list until a match is found or the whole list has been searched.

Linear Search Algorithm Diagram:



As you can see in the diagram above, we have an integer array data structure with some values. We want to search for the **value (98)** which is at **5th** position in this array. Since we are performing the linear search algorithm we start from the beginning of the array and check for matching values till we find a match.

Algorithm to perform Linear Search:

1. Take the input array `arr[]` from user.
2. Take element(**x**) you want to search in this array from user.
3. Set flag variable as **-1**
4. LOOP : `arr[start] -> arr[end]`
 1. if match found i.e `arr[current_position] == x` then
 - Print "Match Found at position" `current_position`.
 - `flag = 0`
 - abort
5. After loop check flag variable.
 1. **if** `flag == -1`
 - print "No Match Found"
6. STOP

The Complexity of Linear Search Algorithm:

You have three different complexities faced while performing Linear Search Algorithm, they are mentioned as follows.

1. Best Case
2. Worst Case
3. Average Case

You will learn about each one of them in a bit more detail.

Best Case Complexity:

- The element being searched could be found in the first position.
- In this case, the search ends with a single successful comparison.
- Thus, in the best-case scenario, the linear search algorithm performs $O(1)$ operations.

Worst Case Complexity:

- The element being searched may be at the last position in the array or not at all.
- In the first case, the search succeeds in 'n' comparisons.
- In the next case, the search fails after 'n' comparisons.
- Thus, in the worst-case scenario, the linear search algorithm performs $O(n)$ operations.

Average Case Complexity:

- When the element to be searched is in the middle of the array, the average case of the Linear Search Algorithm is $O(n)$.
- Next, you will learn about the Space Complexity of Linear Search Algorithm.

Space Complexity of Linear Search Algorithm:

The linear search algorithm takes up no extra space; its space complexity is $O(n)$ for an array of n elements.

Implementation of Linear Search:

```
#include<iostream>

using namespace std;

int main()
{
    int myArray[10];
    int x ;
    int chk = -1;
    for(int i = 0 ; i < 7 ; i++)
    {
        cout<<"Index = "<<i<<endl;
        cin>>myArray[i];
    }
    cout<<"Which value do you wanna search:";
    cin>>x;
    for(int i = 0 ; i < 7 ; i++)
    {
        if(myArray[i]==x)
        {
            cout<<"Number is found on Index "<<i<<endl;
```

```

        chk++;

        break;

    }

}

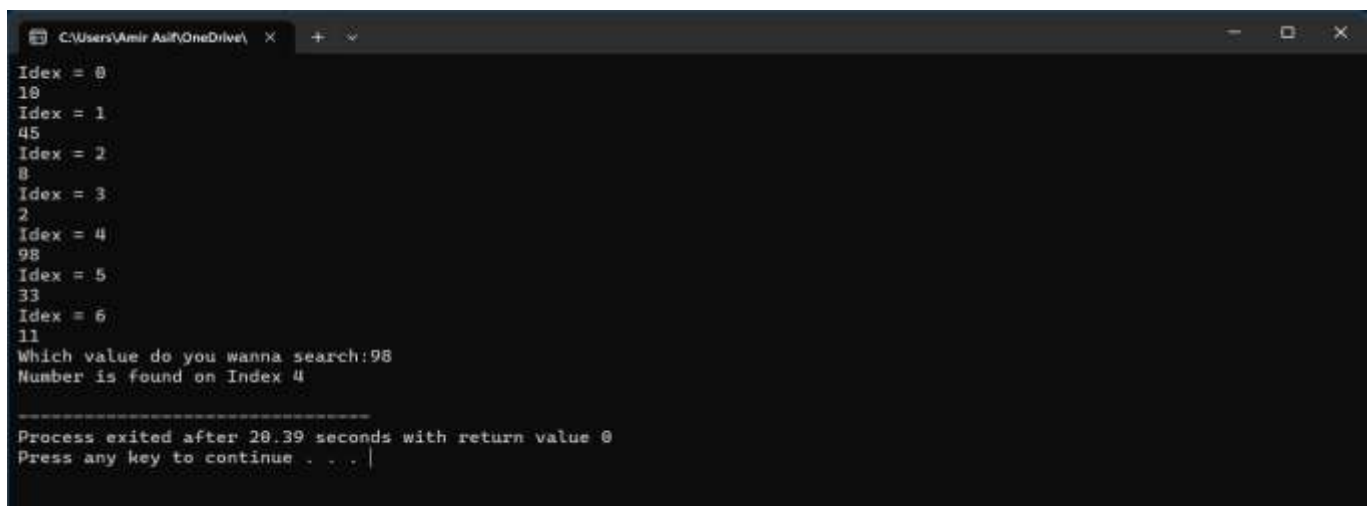
if(chk==-1)
{

    cout<<"Index not found"<<endl;

}

}

```



```

C:\Users\Amir Asif\OneDrive\ >
Index = 0
18
Index = 1
45
Index = 2
8
Index = 3
2
Index = 4
98
Index = 5
33
Index = 6
11
Which value do you wanna search:98
Number is found on Index 4

=====
Process exited after 28.39 seconds with return value 0
Press any key to continue . . .

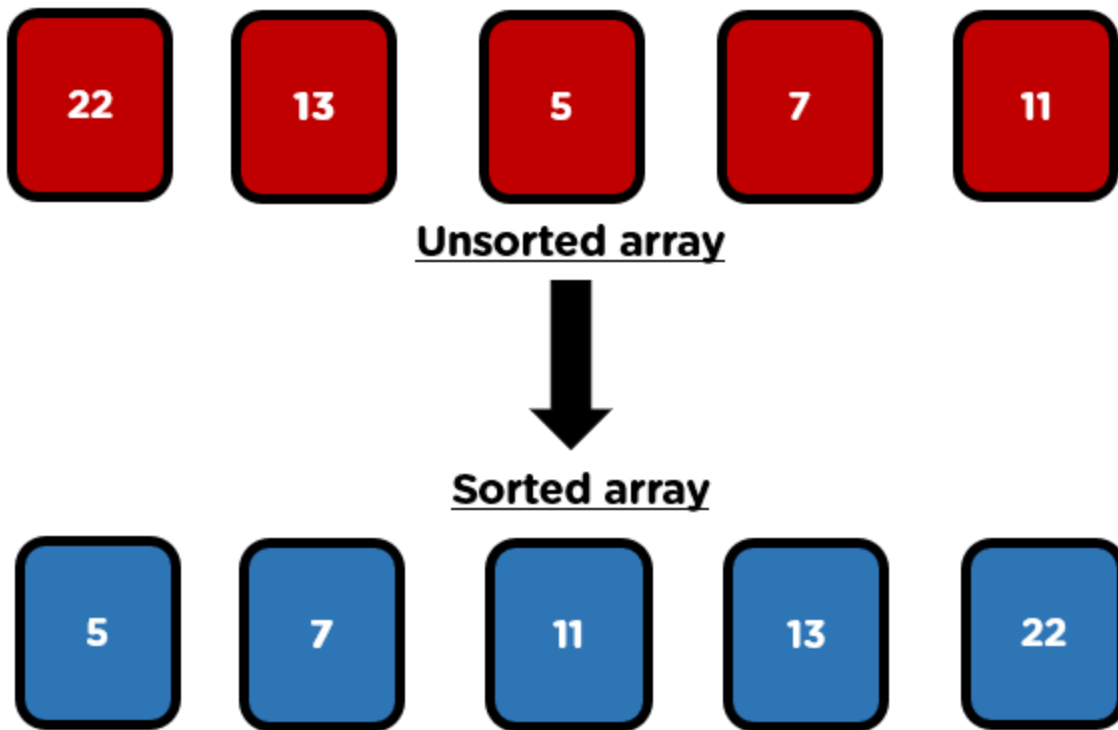
```

What is Sorting in C++?

Sorting in C++ is a concept in which the elements of an array are rearranged in a logical order. This order can be from lowest to highest or highest to lowest. Sorting an unsorted array helps to solve many problems such as searching for the minimum or maximum element, etc.

Arranging things in a sorted manner makes it easier to analyze and search for a particular element among the collection of elements.

For example, In the below example, we are sorting or arranging the elements of an unsorted array in ascending order, i.e., from lowest to highest.



Types of Sorting Techniques:

There are various types of sorting techniques in C++. We will be learning the most popular ones in this article.

- Bubble sort
- Selection sort
- Insertion sort
- Quick sort
- Merge sort

Program 3: Write a C++ program for Bubble Sort:

Bubble Sort:

- Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.
- This algorithm is suitable for small data sets.
- Its average and worst-case complexity are of (n^2) where n is the number of items.
- It is known as bubble sort, because with every complete iteration the largest element bubbles up towards the last place or the highest index just like a water bubble rises to the water surface.

Working

- Step 1 – Starting with the first element (index = 0), compare the current element with the next element of the array.
- Step 2 – If the current element is greater than the next element of the array, swap them.
- Step 3 – If the current element is less than the next element, move to the next element.
- Step 4 – Repeat Step 1 till the list is sorted.

Algorithm to perform Bubble Sort:

1. declare variables – i, j
2. loop : $i = 0$ to $n - 1$ // outer loop
 1. loop : $j = 0$ to $n - i - 1$ // inner loop
 1. if ($a[j] > a[j+1]$) then
 1. swap $a[j]$ & $a[j+1]$
 2. end loop // inner loop
3. end loop // outer loop

Implementation of Bubble Sort:

```
#include<iostream>
using namespace std;

void printArray(int Arr[])
{
    for(int i = 0 ; i < 5 ; i ++ )
    {
        cout << Arr[i]<<" ";
    }
}

void bubbleSort(int A[],int n)
{
    for(int i = 0 ; i < n-1 ; i++)
    {
        for(int j = 0 ; j < n-1-i ; j++)
        {
            if(A[j]>A[j+1])
            {
                int temp = A[j];
```

```

        A[j]=A[j+1];
        A[j+1]=temp;
    }

}

}

int main()
{
    int a[5];
    int n = 5;
    for(int i = 0 ; i < 5 ; i ++ )
    {
        cin >> a[i];
    }
    cout<<"\nUnSorted Array"<<endl;
    printArray(a);
    bubbleSort(a,n);
    cout<<endl;
    printArray(a);
}

```

```

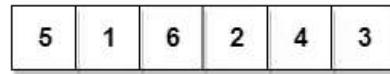
C:\Users\Amir Asif\OneDrive\...
98
7
99
1
100

UnSorted Array: 98 7 99 1 100
Sorted Array: 1 7 98 99 100
-----
Process exited after 9.275 seconds with return value 0
Press any key to continue . . .

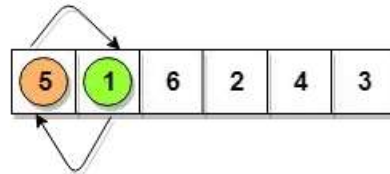
```

Bubble Sort Algorithm Diagram:

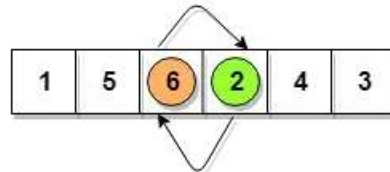
5 > 1
so interchange



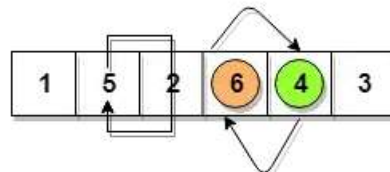
5 < 6
No swapping



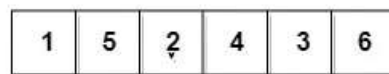
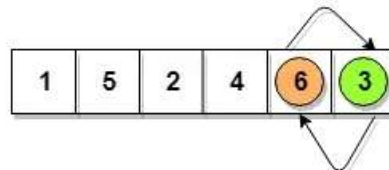
6 > 2
so interchange



6 > 4
so interchange



6 > 3
so interchange



This is first insertion

similarly, after all the iterations, the array gets sorted

Program 4: Write a C++ program for Selection Sort:

Selection Sort:

- Selection sort is a sorting algorithm, specifically an **in-place comparison sort**.
- It has **$O(n^2)$** time complexity, making it inefficient on large lists.

- The algorithm divides the input list into two parts: the sub-list of items already sorted, which is built up from left to right at the front (left) of the list, and the sub-list of items remaining to be sorted that occupy the rest of the list.
- Initially, the sorted sub-list is empty, and the unsorted sub-list is the entire input list. The algorithm proceeds by finding the smallest (or largest, depending on sorting order) element in the unsorted sub-list, exchanging (swapping) it with the leftmost unsorted element (putting it in sorted order), and moving the sub-list boundaries one element to the right.

Selection Sort Algorithm:

- for i=0 to n-1 // outer for loop
 - min = i // set min value to i
 - for j=i+1 to n // inner loop
 - if arr[j] less than arr[min] then // check which element is smaller
 - min=j // store index of smallest element to min
 - end for // inner loop
 - if min not equal to i then // swap if min does not match to i
 - swap arr[min] with arr[i] // swapping
- end for // outer for loop

Implementation of Selection Sort:

```
#include<iostream>
using namespace std;

void selectionSort(int arr[])
{
    for(int i = 0 ; i < 4 ; i++)
    {
        int min = i;
        for(int j = i + 1 ; j < 5 ; j++)
        {
            if(arr[j]<arr[min])
            {
                min=j;
            }
        }
        if(min!=i)
        {
            int temp = arr[min];
            arr[min] = arr [i];
            arr[i] = temp;
        }
    }
}

int main()
{
```

```

int a[5];
for(int i = 0 ; i < 5 ; i++)
{
    cout << "Enter the Value for index " << i << endl;
    cin >> a[i];
}
cout << "\n\nUnsorted Array\n" << endl;
for(int i = 0 ; i < 5 ; i++)
{
    cout << a[i] << " ";
}
selectionSort(a);

cout<<"\n\nSorted Array "<<endl;

for(int i = 0 ; i < 5 ; i++)
{
    cout<<a[i]<<" ";
}
return 0;
}

```

```

C:\Users\Amir Asif\OneDrive\
Enter the Value for index 0
6
Enter the Value for index 1
5
Enter the Value for index 2
4
Enter the Value for index 3
8
Enter the Value for index 4
7

Unsorted Array

6 5 4 8 7

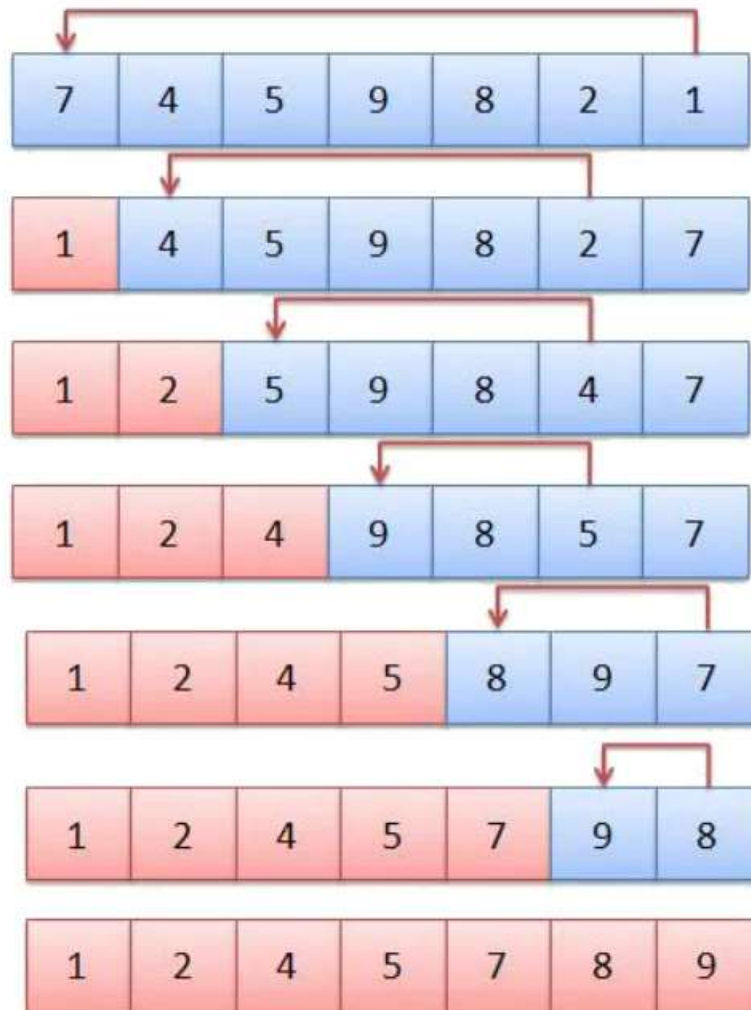
Sorted Array

4 5 6 7 8

=====
Process exited after 10.08 seconds with return value 0
Press any key to continue . . . |

```

Selection Sort Algorithm Diagram:



Program 5: Write a C++ program for Insertion Sort:

Insertion Sort:

- Insertion sort is a simple sorting algorithm that works the way we sort playing cards in our hands.
- Insertion sort is a simple sorting algorithm that builds the final sorted array (or list) one item at a time.
- Time Complexity: $O(n^2)$
- Efficient for (quite) small data sets, much like other quadratic sorting algorithms
- More efficient in practice than most other simple quadratic (i.e., $O(n^2)$) algorithms such as selection sort or bubble sort

Working:

- Step 1 – Pick the next element.
- Step 2 – Compare with all elements in the sorted sub-list on the left.
- Step 3 – Shift all the elements in the sorted sub-list that is greater than the value to be sorted.
- Step 4 – Insert the value.
- Step 5 – Repeat until the list is sorted.

Insertion Sort Algorithm:

1. declare variables – i, key, j
2. loop : i = 1 to n – 1 // outer loop
 1. key = a[i] //pick the next element
 2. j = i – j; // decrement j value
 3. loop : (j>=0 && a[j]>key) // inner loop
 - arr[j+1] = arr[j]
 - j = j – 1
 4. end loop // outer loop
 5. arr[j+1] = key
3. end loop // outer loop

Implementation of Insertion Sort:

```
#include<iostream>
using namespace std;

void printArray(int A[])
{
    for(int i = 0 ; i < 5 ; i++)
    {
        cout<<A[i]<<" ";
    }
}

void insertionSort(int A[])
{
    int j , temp ;
    for(int i = 1 ; i < 5 ; i++)
    {
        temp=A[i];
        j=i-1;
        while(j>=0&&A[j]>temp)
        {
            A[j+1]=A[j];
            j--;
        }
        A[j+1]=temp;
    }
}

int main()
```

```

{
    int a[5];

    for(int i = 0 ; i < 5 ; i++)
    {
        cin>>a[i];
    }
    cout<<"\nUnsorted Array\n";
    printArray(a);
    insertionSort(a);
    cout<<"\nsorted Array\n";
    printArray(a);
}

```

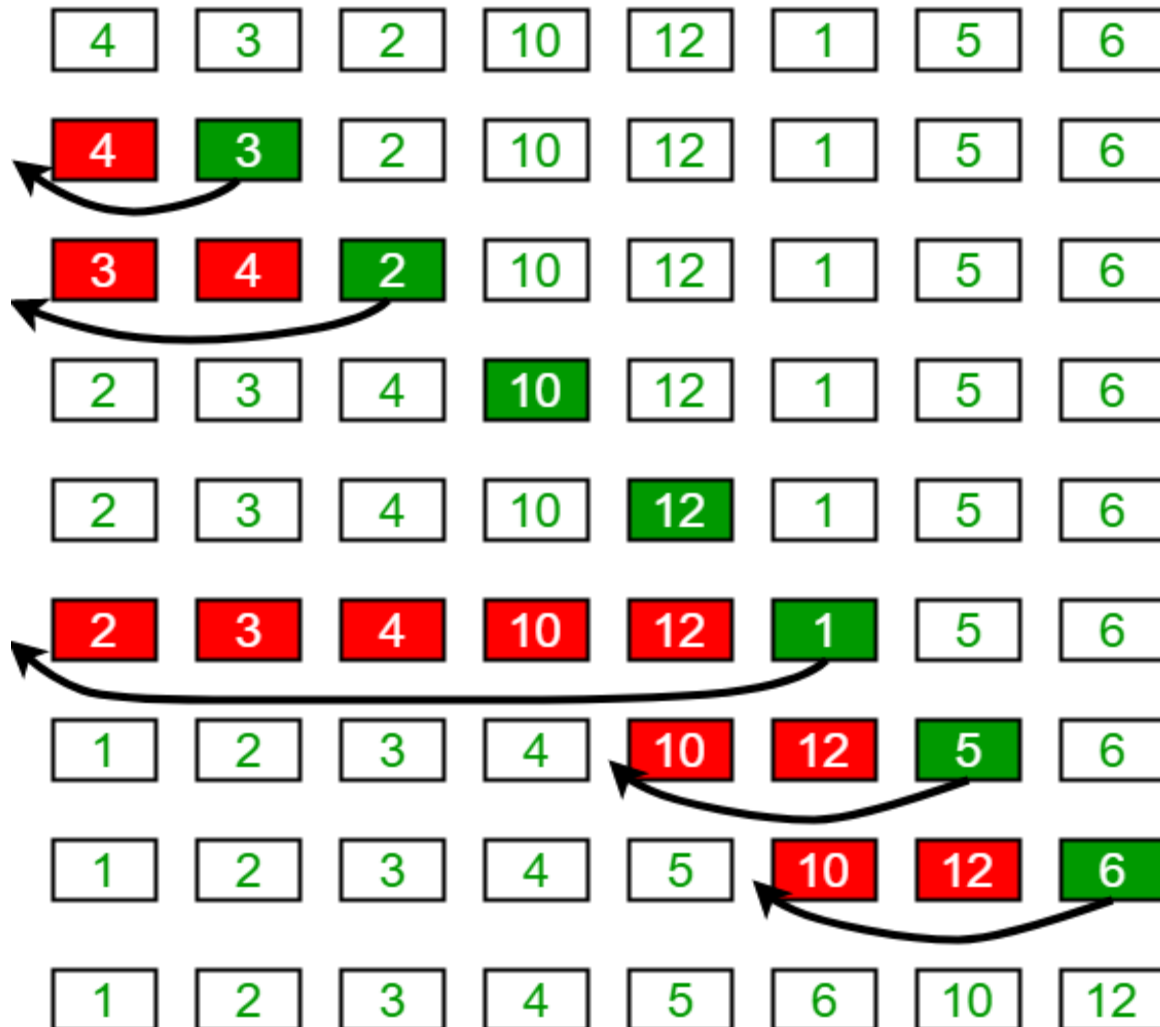
```

C:\Users\Amir Asif\OneDrive\
9
8
7
6
5
Unsorted Array
9 8 7 6 5
sorted Array
5 6 7 8 9
=====
Process exited after 9.046 seconds with return value 0
Press any key to continue . . .

```

Insertion Sort Algorithm:

Insertion Sort Execution Example



What is Recursion in C++?

Recursion is a method in C++ which calls itself directly or indirectly until a suitable condition is met. In this method, we repeatedly call the function within the same function, and it has a base case and a recursive condition. The recursive condition helps in the repetition of code again and again, and the base case helps in the termination of the condition.

If there is no base case in the recursive function, the recursive function will continue to repeat continuously.

Program 6: Write a C++ program for Merge Sort:

Description: Merge Sort Algorithm is a **Divide & Conquer** algorithm. It divides input array in two halves, calls itself for the two halves(recursively) and then merges the two sorted halves. A separate **merge ()** function is used for merging two halves. Merge sort is one of the **most efficient sorting algorithms**.

Time Complexity: $O(n\log(n))$

Working:

There are 3 Phases (4 major steps) in the Merge Sort Algorithm:

1. Division Phase – Divide the array(list) into 2 halves by finding the midpoint of the array(list).
 1. Midpoint (m) = (left + right)/ 2
 2. *Here left is the starting index & right is the last index of the array(list)*
2. Recursion Phase –
 1. Call Merge Sort on the left sub-array (sub-list)
 2. Call Merge Sort on the right sub-array (sub-list)
3. Merge Phase –
 1. Call merge function to merge the divided sub-arrays back to the original array.
 2. Perform sorting of these smaller sub arrays before merging them back.

Merge Sort Algorithm (Pseudo Code)

```
mergeSort(arr[],l,r)  //arr is array, l is left, r is right
{
    if(l<r)
    {
        midpoint = (l+r)/2
        mergeSort(arr,l,m)
        mergeSort(arr,m+1,r)
        merge(arr,l,m,r)
    }
}
```

Implementation of Merge Sort:

```
#include <iostream>

using namespace std;

void merge(int arr[], int l, int m, int r) {

    int i = l;

    int j = m + 1;

    int k = l;

    /* create temp array */

    int temp[5];

    while (i <= m && j <= r) {

        if (arr[i] <= arr[j]) {

            temp[k] = arr[i];

            i++;

            k++;

        } else {

            temp[k] = arr[j];

            j++;

            k++;

        }

    }

    /* Copy the remaining elements of first half, if there are any */

    while (i <= m) {

        temp[k] = arr[i];

        i++;

        k++;

    }

    /* Copy the remaining elements of second half, if there are any */

    while (j <= r) {
```

```
temp[k] = arr[j];  
j++;  
k++;  
}
```

```
/* Copy the temp array to original array */
```

```
for (int p = l; p <= r; p++) {  
    arr[p] = temp[p];  
}  
}
```

```
/* l is for left index and r is right index of the  
sub-array of arr to be sorted */
```

```
void mergeSort(int arr[], int l, int r) {
```

```
    if (l < r) {
```

```
        // find midpoint
```

```
        int m = (l + r) / 2;
```

```
        // recursive mergesort first and second halves
```

```
        mergeSort(arr, l, m);
```

```
        mergeSort(arr, m + 1, r);
```

```
        // merge
```

```
        merge(arr, l, m, r);
```

```
    }
```

```
}
```

```
int main() {
```

```
    int myarray[5];
```

```
    //int arr_size = sizeof(myarray)/sizeof(myarray[0]);
```

```
    int arr_size = 5;
```

```
    cout << "Enter 5 integers in any order: " << endl;
```

```

for (int i = 0; i < 5; i++) {
    cin >> myarray[i];
}

cout << "Before Sorting" << endl;
for (int i = 0; i < 5; i++) {
    cout << myarray[i] << " ";
}

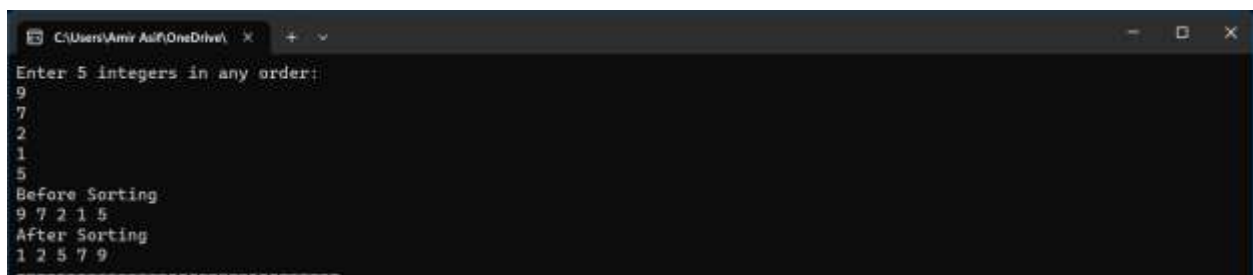
cout << endl;

mergeSort(myarray, 0, (arr_size - 1)); // mergesort(arr,left,right) called

cout << "After Sorting" << endl;
for (int i = 0; i < 5; i++) {
    cout << myarray[i] << " ";
}

return 0;
}

```



```

C:\Users\Amir Asif\OneDrive\ >
Enter 5 integers in any order:
9
7
2
1
5
Before Sorting
9 7 2 1 5
After Sorting
1 2 5 7 9

```

Merge Sort Algorithm Diagram:

1. Divide the array into two parts

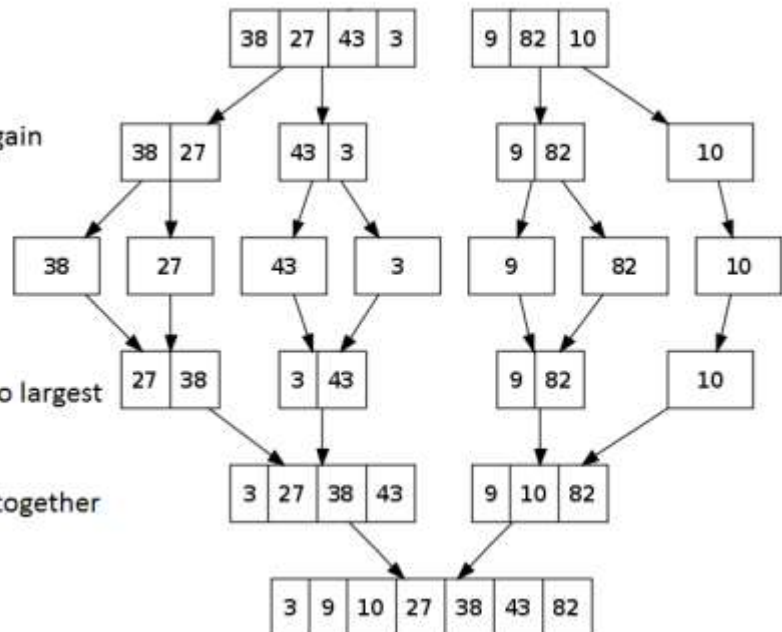
2. Divide the array into two parts again

3. Break each element into single parts

4. Sort the elements from smallest to largest

5. Merge the divided sorted arrays together

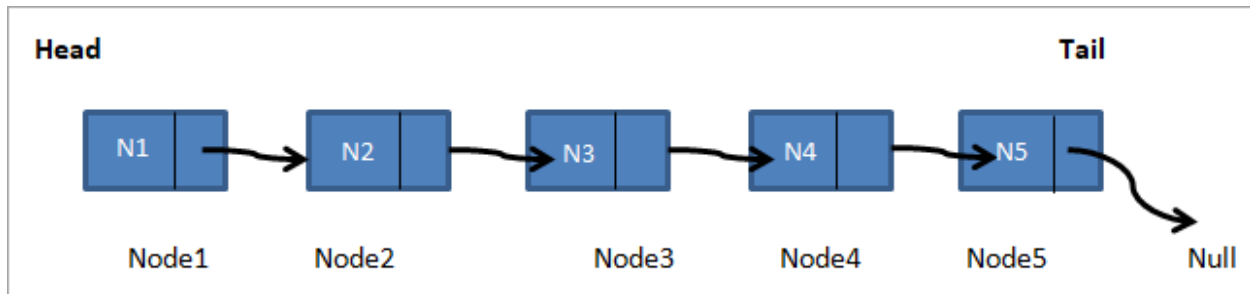
6. The array has been sorted



Program 7: Write a C++ program for Single Link List and create a menu.

1. **Create LL**
2. **Insert At Head**
3. **Insert At Tail**
4. **Insert After Position**
5. **Insert Before Position**
6. **Del Head**
7. **Del Tail**
8. **Del By Position**
9. **Del By Value**
10. **Sorting**
11. **Change head to tail**
12. **Change Tail to Head**
13. **Swap node**
14. **Display**
15. **Reverse**

The following diagram shows the structure of a singly linked list.



As shown above, the first node of the linked list is called “head” while the last node is called “Tail”. As we see, the last node of the linked list will have its next pointer as null since it will not have any memory address pointed to.

Since each node has a pointer to the next node, data items in the linked list need not be stored at contiguous locations. The nodes can be scattered in the memory. We can access the nodes anytime as each node will have an address of the next node.

We can add data items to the linked list as well as delete items from the list easily. Thus, it is possible to grow or shrink the linked list dynamically. There is no upper limit on how many data items can be there in the linked list. So, if memory is available, we can have as many data items added to the linked list.

Apart from easy insertion and deletion, the linked list also doesn’t waste memory space as we need not specify beforehand how many items we need in the linked list. The only space taken by linked list is for storing the pointer to the next node that adds a little overhead.

Next, we will discuss the various operations that can be performed on a linked list.

Operations:

Just like the other data structures, we can perform various operations for the linked list as well. But unlike arrays, in which we can access the element using subscript directly even if it is somewhere in between, we cannot do the same random access with a linked list.

To access any node, we need to traverse the linked list from the start and only then we can access the desired node. Hence accessing the data randomly from the linked list proves to be expensive.

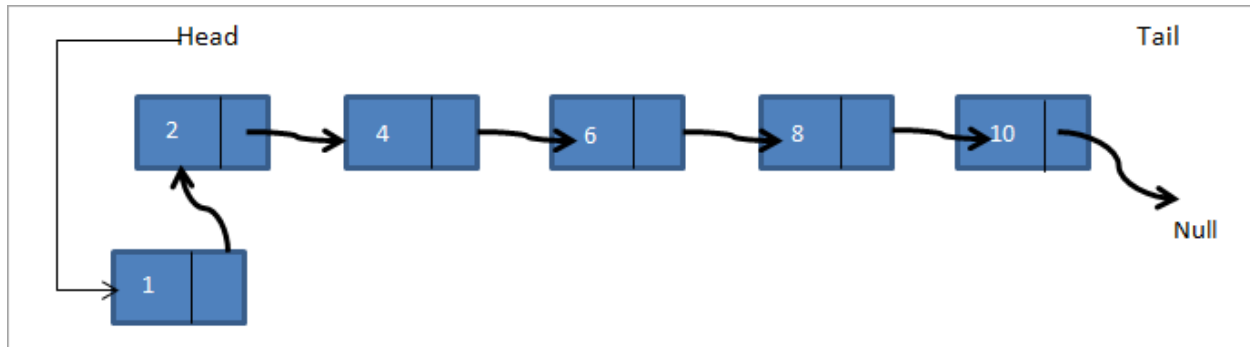
We can perform various operations on a linked list as given below:

Insertion:

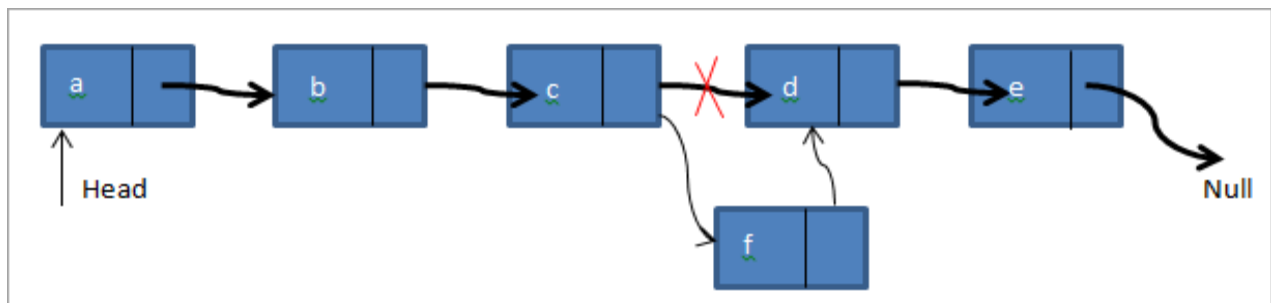
Insertion operation of linked list adds an item to the linked list. Though it may sound simple, given the structure of the linked list, we know that whenever a data item is added to the linked list, we need to change the next pointers of the previous and next nodes of the new item that we have inserted.

The second thing that we must consider is the place where the new data item is to be added.

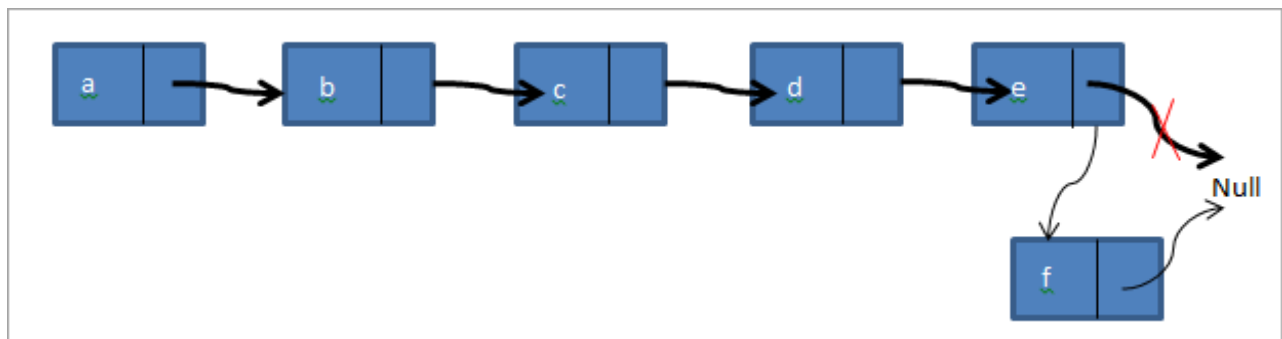
At the beginning of the linked list:



After the given Node:



At the end of the linked list:



Deletion:

Like insertion, deleting a node from a linked list also involves various positions from where the node can be deleted. We can delete the first node, last node, or a random kth node from the linked list.

After deletion, we need to adjust the next pointer and the other pointers in the linked list appropriately to keep the linked list intact.

Implementation of code:

```
#include <iostream>

using namespace std;

void createLL();

void display();

int Length();

void insertAtHead();

void insertAtTail();

void insertAfterPosition();

void insertBeforePosition();

void delHead();

void delTail();

void delAfterPosition();

void delByValue();

void sortLL();

void sortLLL();

void tailtohead();

void headtotail();

void swapnode();

void reserve();

struct node

{

    int value;
```

```

        node* next;

};

int main()

{
    int x , y ;

    do
    {

        cout << "1: Create Link List " << endl;

        cout << "2: Insert New Head " << endl;

        cout << "3: Insert New Tail " << endl;

        cout << "4: Insert After Position " << endl;

        cout << "5: Insert Before Position " << endl;

        cout << "6: Delete Head Node " << endl;

        cout << "7: Delete Tail Node " << endl;

        cout << "8: Delete At position " << endl;

        cout << "9: Delete by Value " << endl;

        cout << "10: Sorting LL in Ascending order" << endl;

        cout << "11: Sorting LL in Descending order" << endl;

        cout << "12: Convert Tail into head " << endl;

        cout << "13: Convert head into tail " << endl;

        cout << "14: Swap 2 nodes " << endl;

        cout << "15: Reverse a Link List " << endl;

        cout << "16: Get length of Link List " << endl;

        cout << "17: Display Link List " << endl;

        cout << "\n-----Press any option Given Above-----: " ;
    }
}

```

```
cin >> x ;
```

```
switch(x)
```

```
{
```

```
    case 1:
```

```
        {
```

```
            createLL();
```

```
            break;
```

```
        }
```

```
    case 2:
```

```
        {
```

```
            insertAtHead();
```

```
            break;
```

```
        }
```

```
    case 3:
```

```
        {
```

```
            insertAtTail();
```

```
            break;
```

```
        }
```

```
    case 4:
```

```
        {
```

```
            insertAfterPosition();
```

```
            break;
```

```
        }
```

```
    case 5:
```

```
        {  
            insertBeforePosition();  
            break;  
        }  
case 6:  
    {  
        delHead();  
        break;  
    }  
case 7:  
    {  
        delTail();  
        break;  
    }  
case 8:  
    {  
        delAfterPosition();  
        break;  
    }  
case 9:  
    {  
        delByValue();  
        break;  
    }  
case 10:
```

```
    {  
        sortLL();  
        break;  
    }  
case 11:  
    {  
        sortLLL();  
        break;  
    }  
case 12:  
    {  
        tailtohead();  
        break;  
    }  
case 13:  
    {  
        headtotail();  
        break;  
    }  
case 14:  
    {  
        swapnode();  
        break;  
    }  
case 15:
```

```

        {
            reserve();

            break;
        }
case 16:
    {
        cout<<"Length of current LL : "<< Length() << endl;

        break;
    }
case 17:
    {
        cout<<"Link List: ";

        display();

        break;
    }
default:
    {
        cout << "Invalid choice " << endl;
    }
}

cout << "\nPress 0 for exit and any other key for Main Menu : ";

cin >> y;

//system("cls");

}while(y!=0);

```

```

        return 0;
    }

    node* head = NULL;

    void createLL()
    {
        int x ;

        do

        {

            node* newnode = (struct node*) malloc (sizeof ( struct node ));

            cout << "Enter the newNode value: ";

            cin >> newnode->value;

            newnode->next = NULL;

            if(head == NULL)

            {

                head = newnode;

            }

            else

            {

                node* temp = head;

                while(temp->next != NULL)

                {

                    temp = temp->next;

                }

```

```

        temp->next = newnode;

    }

    cout << "Press 0 for exit and any other key for make a another newNode : ";

    cin >> x ;

    }while(x != 0);

}

void display()

{

    node* temp = head ;

    if(head==NULL)

    {

        cout<<"List is empty "<<endl;

        return;

    }

    while (temp != NULL )

    {

        cout << temp->value << " ";

        temp = temp->next;

    }

    cout << endl;

}

int Length()

{

    node* temp = head ;

    int size = 0;

```

```

while (temp != NULL )
{
    size++;

    temp = temp->next;

}

cout << endl;

return size;

}

void insertAtHead()
{
    node* newnode = (struct node*) malloc (sizeof ( struct node ));

    cout << "Enter the new head Node value: ";

    cin >> newnode->value;

    newnode->next = NULL;

    if(head == NULL)
    {
        head = newnode;
    }

    else
    {
        newnode->next = head;

        head = newnode;
    }

}

void insertAtTail()

```

```

{

    node* newnode = (struct node*) malloc (sizeof ( struct node ));

    cout << "Enter the new tail Node value: ";

    cin >> newnode->value;

    newnode->next = NULL;

    if(head == NULL)

    {

        head = newnode;

        return;

    }

    node* temp = head;

    while(temp->next != NULL)

    {

        temp = temp->next;

    }

    temp->next = newnode;

}

void insertAfterPosition()

{

    int pos;

    int i = 1;

    int l = Length();

    node* temp = head;

    cout<<"Enter the position you wanna insert (After): ";

    cin>>pos;

```

```

if(pos<1 || pos>l)
{
    cout<<"Invalid Position "<<endl;
    return;
}

while(i < pos)
{
    temp = temp->next;
    i++;
}

if(temp->next == NULL)
{
    insertAtTail();
    return;
}

node* newnode = (struct node*) malloc (sizeof ( struct node ));

cout << "Enter the new Node value: ";

cin >> newnode->value;

newnode->next=temp->next;

temp->next = newnode;
}

void insertBeforePosition()
{
    int pos;

    int i = 1;

```

```

int l = Length();

node* temp = head;

cout<<"Enter the position you wanna insert (Before): ";

cin>>pos;

if(pos<1 || pos>l)

{

    cout<<"Invalid Position "<<endl;

    return;

}

if(pos == 1)

{

    insertAtHead();

    return;

}

while(i < pos - 1)

{

    temp = temp->next;

    i++;

}


node* newnode = (struct node*) malloc (sizeof ( struct node ));

cout << "Enter the new Node value: ";

cin >> newnode->value;

newnode->next=temp->next;

temp->next = newnode;

```

```
}
```

```
void delHead()
```

```
{
```

```
    node* temp = head;
```

```
    head = head->next;
```

```
    delete temp;
```

```
}
```

```
void delTail()
```

```
{
```

```
    node* temp = head;
```

```
    node* prev ;
```

```
    while(temp->next!=NULL)
```

```
    {
```

```
        prev = temp;
```

```
        temp = temp->next;
```

```
    }
```

```
    prev->next=NULL;
```

```
    delete temp;
```

```
}
```

```
void delAfterPosition()
```

```
{
```

```
    int i = 1;
```

```
    node* temp = head;
```

```
    int pos;
```

```
    cout << "Enter position of node you wanna delete: ";
```

```
cin >> pos;

int l = Length();

if(pos < 1 || pos > l)

{

    cout << "Invalid Position " << endl;

    return;

}

if(pos == 1)

{

    delHead();

    return;

}

while(i < pos - 1 )

{

    temp = temp->next;

    i++;

}

if(temp->next == NULL)

{

    delTail();

    return;

}

node*nextnode = temp->next;

temp->next=nextnode->next;

delete nextnode;
```

```

}

void delByValue()
{
    int x;

    cout << "Enter value you wanna Delete: ";

    cin >> x;

    node* temp = head;

    node* prev = temp;

    while(temp != NULL)
    {
        if(temp->value == x)
        {
            if(head==temp)
            {
                delHead();

                return;
            }

            else
            {
                prev->next = temp->next;

                delete temp;

                return;
            }
        }
    }
}

```

```

        else
        {
            prev=temp;
            temp=temp->next;
        }
    }

    cout<<"Value not Found "<<endl;
}

void sortLL()
{
    node* temp = head ;
    node* nextnode ;
    int swap;
    while(temp!=NULL)
    {
        nextnode=temp->next;
        while(nextnode != NULL)
        {
            if(temp->value > nextnode->value)
            {
                swap=temp->value;
                temp->value=nextnode->value;
                nextnode->value=swap;
            }
            nextnode=nextnode->next;

```

```

        }

        temp=temp->next;

    }

}

void sortLLL()

{

    node* temp = head ;

    node* nextnode ;

    int swap;

    while(temp!=NULL)

    {

        nextnode=temp->next;

        while(nextnode != NULL)

        {

            if(temp->value < nextnode->value)

            {

                swap=temp->value;

                temp->value=nextnode->value;

                nextnode->value=swap;

            }

            nextnode=nextnode->next;

        }

        temp=temp->next;

    }

}

```

```
void tailtohead()
{
    node* temp = head;

    node* prev = NULL;

    while(temp->next!=NULL)
    {
        prev = temp;
        temp=temp->next;
    }

    temp->next=head;

    head=temp;

    prev->next=NULL;
}
```

```
void headtotail()
{
    node*temp=head;

    while(temp->next!=NULL)
    {
        temp=temp->next;
    }

    temp->next=head;

    head=head->next;

    temp->next->next=NULL;
}
```

```
void swapnode()
```

```

{

    int x ;

    int y ;

    cout<<"Enter the value of first node: ";

    cin>>x;

    cout<<"Enter the value of second node: ";

    cin>>y;

    if(x==y)

    {

        return;

    }

    node* currentx = head;

    node* prevx = NULL;

    while(currentx->next !=NULL && currentx->value != x)

    {

        prevx=currentx;

        currentx = currentx->next;

    }

    node* currenty = head;

    node* prevy = NULL;

    while(currenty->next!=NULL && currenty->value!=y)

    {

        prevy=currenty;

        currenty=currenty->next;

    }
}

```

```
if(currentx==NULL || currenty==NULL)
{
    cout<<"Value not found "<<endl;
    return;
}
if(prevx==NULL)
{
    head=currenty;
}
else
{
    prevx->next=currenty;
}
if(prevy==NULL)
{
    head=currentx;
}
else
{
    prevy->next=currentx;
}
node *temp = currenty->next;
currenty->next=currentx->next;
currentx->next=temp;
}
```

```

void reserve()
{
    node* prev = NULL;

    node* current = head;

    node* nextnode = head;

    while(nextnode!=NULL)
    {
        nextnode=nextnode->next;

        current->next=prev;

        prev=current;

        current=nextnode;
    }

    head=prev;
}

```

Program 8: Merge 2 already sorted Link List.

```

#include<iostream>

using namespace std;

struct node
{
    int value;

    node*next;
};

node*head1=NULL;

```

```

node*head2=NULL;

void createLL1()
{
    node* newNode = (struct node*)malloc(sizeof(struct node));

    cout<<"Enter the new node (Value): ";

    cin>>newNode->value;

    newNode->next=NULL;

    if(head1==NULL)
    {
        head1=newNode;
    }
    else
    {
        node*temp=head1;

        while(temp->next!=NULL)
        {
            temp=temp->next;
        }

        temp->next=newNode;
    }
}

void createLL2()
{
    node* newNode = (struct node*)malloc(sizeof(struct node));

    cout<<"Enter the new node (Value): ";

```

```
cin>>newNode->value;

newNode->next=NULL;

if(head2==NULL)

{

    head2=newNode;

}

else

{

    node*temp=head2;

    while(temp->next!=NULL)

    {

        temp=temp->next;

    }

    temp->next=newNode;

}

}

void print1()

{

    node* t = head1;

    while(t!=NULL)

    {

        cout<<t->value<<" ";

        t=t->next;

    }

    cout<<endl;
```

```
}
```

```
void print2()
```

```
{
```

```
    node* t = head2;
```

```
    while(t!=NULL)
```

```
    {
```

```
        cout<<t->value<<" ";
```

```
        t=t->next;
```

```
    }
```

```
    cout<<endl;
```

```
}
```

```
node* merge()
```

```
{
```

```
    node* l1 = head1;
```

```
    node* l2 = head2;
```

```
    node* sorting;
```

```
    node* newhead;
```

```
    if(l1 == NULL) { return l2;}
```

```
    if(l2 == NULL) { return l1;}
```

```
    if(l1 && l2)
```

```
    {
```

```
        if(l1->value <= l2->value)
```

```
        {
```

```
            sorting = l1;
```

```
            l1 = sorting->next;
```

```

    }

    else

    {

        sorting = l2;

        l2 = sorting->next;

    }

}

newhead=sorting;

while(l1 && l2)

{

    if(l1->value <= l2->value)

    {

        sorting->next=l1;

        sorting=l1;

        l1=sorting->next;

    }

    else

    {

        sorting->next=l2;

        sorting=l2;

        l2=sorting->next;

    }

}

if(l1==NULL){sorting->next=l2;    }

```

```
        if(l2==NULL){sorting->next=l1;    }

        return newhead;

    }
```

```
int main()

{

    cout<<"First LL ... "<<endl;

    createLL1();

    createLL1();

    createLL1();

    createLL1();

    print1();

    cout<<"2nd LL ... "<<endl;

    createLL2();

    createLL2();

    createLL2();

    createLL2();

    print2();

    merge();

    cout<<"After merge: ";

    print1();

    return 0;

}
```

```
C:\Users\Amir Asif\OneDrive\ ...
First LL ...
Enter the new node (Value): 1
Enter the new node (Value): 3
Enter the new node (Value): 5
Enter the new node (Value): 6
1 3 5 6
2nd LL ...
Enter the new node (Value): 2
Enter the new node (Value): 4
Enter the new node (Value): 7
Enter the new node (Value): 9
2 4 7 9
After merge: 1 2 3 4 5 6 7 9

=====
Process exited after 24.89 seconds with return value 0
Press any key to continue . . .
```

Program 9: Sort Link List while Taking input.

```
#include<iostream>

using namespace std;

struct node
{
    int value;

    node* next;

};

node* head = NULL;

void create()
{
    node* newNode = (struct node*)malloc(sizeof(struct node));

    cout<<"Enter the value of New Node: ";
```

```
cin>>newNode->value;

newNode->next=NULL;

node*temp;

node*prev;

if(head==NULL)

{

head=newNode;

}

else

{

for(temp = head,prev = head;temp!= NULL && temp->value < newNode->value ; prev =

temp, temp = temp->next);

{

if(temp == head && temp->value > newNode->value)

{

newNode->next=head;

head=newNode;

}

else

{

prev->next=newNode;

newNode->next=temp;

}

}

}
```

```
}

void print()

{

    node* temp = head;

    if(head==NULL)

    {

        cout<<"List is empty "<<endl;

        return;

    }

    while(temp!=NULL)

    {

        cout<<temp->value<<" ";

        temp=temp->next;

    }

    cout<<endl;

}

int main()

{

    create();

    print();

    create();

    print();

    create();

    print();

    create();
```

```

print();

create();

print();

return 0;

}

```

```

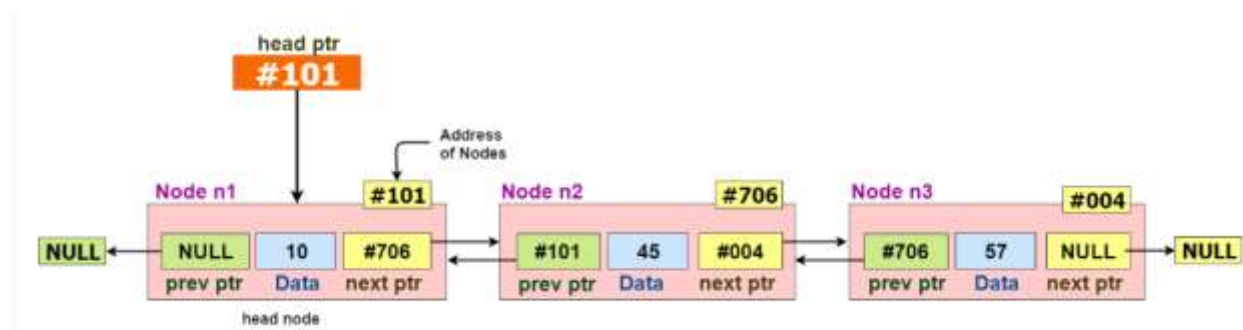
C:\Users\Amir Asif\OneDrive\ x + v
Enter the value of New Node: 6
6
Enter the value of New Node: 4
4 6
Enter the value of New Node: 7
4 6 7
Enter the value of New Node: 1
1 4 6 7
Enter the value of New Node: 2
1 2 4 6 7
-----

```

Program 10: Write a C++ program for double Link List and create a menu.

Description: Doubly Linked list is a type of Linked List Data structure which behaves like a **two-way list/chain**. The reason it is called a two-way list or two-way chain is because we can **traverse this list in two directions**.

- start from the head node to the end.
- go back in the reverse direction.



As you can see from the diagram, each node object has **1 data** field & **2 pointer** fields. The data field contains the actual data whereas the pointer fields (next & previous pointers) point to the next node & previous node in the doubly linked list. Since the nodes are not stored in contiguous memory

locations, these extra pointer fields assist in locating the next & previous nodes in memory. As we have 2 pointers: one pointing to the next node and one pointing to the previous node, we can traverse in 2 directions starting from the head node to the end and vice versa.

Following are the standard Singly Linked List Operations –

- **Traverse** – Iterate through the nodes in the linked list starting from the head node.
- **Append** – Attach a new node (to the end) of a list.
- **Prepend** – Attach a new node (to the beginning) of the list.
- **Insert** – attach a new node to a specific position on the list.
- **Delete** – Remove/Delink a node from the list.

```
#include<iostream>
```

```
using namespace std;
```

```
struct node
```

```
{
```

```
    char value;
```

```
    node*next;
```

```
    node*pre;
```

```
};
```

```
node* head ;
```

```
node* tail ;
```

```
void createDLL()
```

```
{
```

```
    node*newNode ;
```

```
    newNode = (struct node*)malloc(sizeof(struct node));
```

```
    cout<<"Enter Value You Wanna insert in node: ";
```

```
cin>>newNode->value;

newNode->next=NULL;

newNode->pre=NULL;

if(head==NULL)

{

    head= newNode;

    tail= newNode;

}

else

{

    newNode->next=NULL;

    newNode->pre=tail;

    tail->next=newNode;

    tail=newNode;

}

}

void insertAtHead()

{

    node* newNode = (struct node*)malloc(sizeof(struct node));

    cout<<"Enter Value You Wanna insert in new head node: ";
```

```

        cin>>newNode->value;

        newNode->next=NULL;

        newNode->pre=NULL;

        newNode->next = head;

        head->pre=newNode;

        head=newNode;
    }

    void insertAtEnd()

    {

        node* newNode = (struct node*)malloc(sizeof(struct node));

        cout<<"Enter Value You Wanna insert in new end node: ";

        cin>>newNode->value;

        newNode->next=NULL;

        newNode->pre=NULL;

        newNode->pre=tail;

        tail->next=newNode;

        tail=newNode;

    }

    void insertAfterPosition()

    {

```

```
int pos;

int i = 1;

node*temp;

temp= head;

cout<<"Enter position you wanna insert new Node: ";

cin>>pos;

node* newNode = (struct node*)malloc(sizeof(struct node));

cout<<"Enter Value You Wanna insert in new end node: ";

cin>>newNode->value;

if(pos<1)

{

    cout<<"Invalid position";

    return;

}

while(i<pos)

{

    temp=temp->next;

    i++;

}

newNode->pre=temp;
```

```
newNode->next=temp->next;
```

```
temp->next=newNode;
```

```
newNode->next->pre=newNode;
```

```
}
```

```
void insertAtPosition()
```

```
{
```

```
    int pos;
```

```
    int i = 1;
```

```
    node*temp;
```

```
    temp= head;
```

```
    cout<<"Enter position you wanna insert new Node: ";
```

```
    cin>>pos;
```

```
    if(pos<1)
```

```
    {
```

```
        cout<<"Invalid position";
```

```
        return;
```

```
    }
```

```
    if(pos==1)
```

```

{

    insertAtHead();

    return;

}

while(i<pos-1)

{

    temp=temp->next;

    i++;

}

if(temp->next==NULL)

{

    insertAtEnd();

    return;

}

node* newNode = (struct node*)malloc(sizeof(struct node));

cout<<"Enter Value You Wanna insert in new end node: ";

cin>>newNode->value;

newNode->pre=temp;

newNode->next=temp->next;

temp->next=newNode;

```

```
newNode->next->pre=newNode;
```

```
}
```

```
void printForward(node* head)
```

```
{
```

```
    node* treverse ;
```

```
    treverse = head;
```

```
    while(treverse!=NULL)
```

```
    {
```

```
        cout<<treverse->value<<" ";
```

```
        treverse=treverse->next;
```

```
    }
```

```
    cout<<endl;
```

```
}
```

```
void printReverse(node* tail)
```

```
{
```

```
    node* treverse ;
```

```
    treverse = tail;
```

```
    while(treverse!=NULL)
```

```
    {
```

```
        cout<<treverse->value<<" ";

        treverse=treverse->pre;

    }

    cout<<endl;

}
```

```
void delHead()
```

```
{

    node*temp;

    temp=head;

    head=head->next;

    head->pre=NULL;

    delete temp;

}
```

```
void delTail()
```

```
{

    node* temp;

    temp = tail;

    tail=tail->pre;

    tail->next=NULL;

    delete temp;

}
```

```
}
```

```
void delAtPosition()
```

```
{
```

```
    int pos ;
```

```
    int i = 1 ;
```

```
    cout<<"Enter Position of node you wanna Delete: ";
```

```
    cin>>pos;
```

```
    node*temp ;
```

```
    temp = head ;
```

```
    if(pos<1)
```

```
    {
```

```
        cout<<"Invalid Position ";
```

```
        return;
```

```
    }
```

```
    if(pos==1)
```

```
    {
```

```
        delHead();
```

```
    }
```

```
    while(i<pos)
```

```
    {
```

```
        temp=temp->next;

        i++;

    }

    if(temp->next==NULL)

    {

        delTail();

        return;

    }

    temp->pre->next=temp->next;

    temp->next->pre=temp->pre;

    delete temp;

}
```

```
int lengthLL()
```

```
{

    int size = 0;

    node* treverse ;

    treverse = head;

    while(treverse!=NULL)

    {

        size++;

    }

}
```

```
        traverse=traverse->next;

    }

    return size;

}

int main()

{

    createDLL();

    createDLL();

    createDLL();

    createDLL();

    createDLL();

    cout<<"\nLength of Doubly LL is: "<<lengthLL()<<endl;

    printForward(head);

    insertAtHead();

    cout<<"\nLength of Doubly LL is: "<<lengthLL()<<endl;

    printForward(head);

    insertAfterPosition();

    cout<<"\nLength of Doubly LL is: "<<lengthLL()<<endl;

    printForward(head);

    delAtPosition();
```

```

cout<<"\nLength of Doubly LL is: "<<lengthLL()<<endl;

printForward(head);

}

```

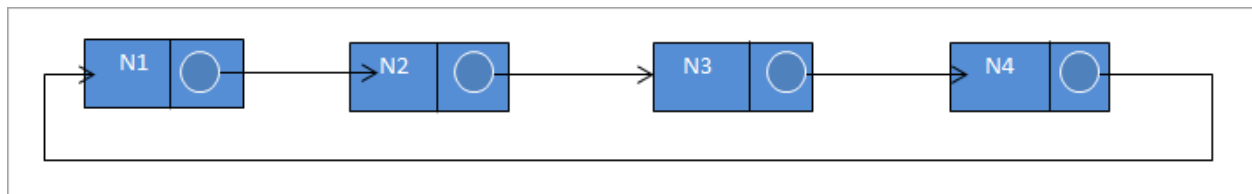
Program 11: Write a C++ program for Circular Link List.

A Complete Overview of Circular Linked List.

A circular linked list is a variation of the linked list. It is a linked list whose nodes are connected in such a way that it forms a circle.

In the circular linked list, the next pointer of the last node is not set to null, but it contains the address of the first node thus forming a circle.

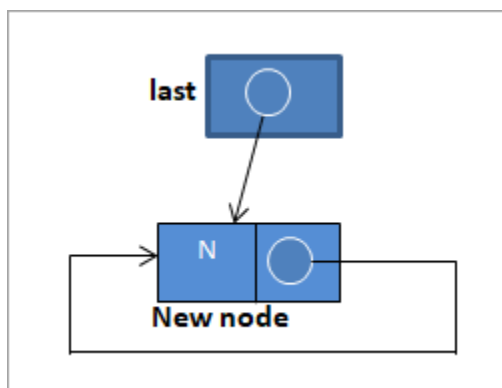
The arrangement shown below is for a singly Linked List.



Insertion:

We can insert a node in a circular linked list either as a first node (empty list), in the beginning, in the end, or in between the other nodes. Let us see each of these insertion operations using a pictorial representation below.

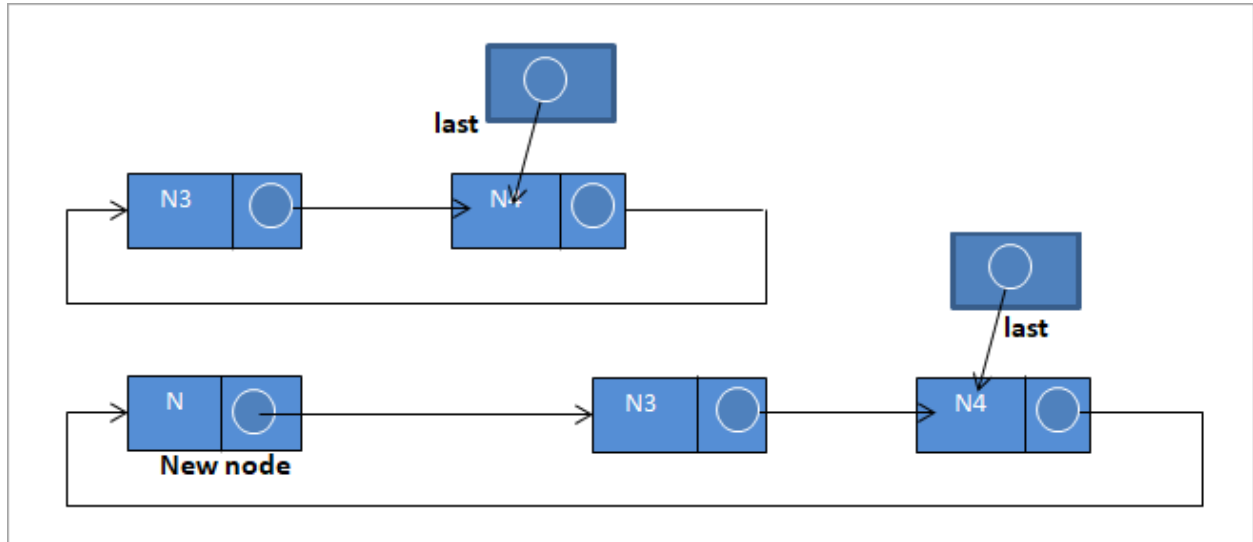
Insert in an empty list:



When there are no nodes in circular list and the list is empty, the last pointer is null, then we insert a new node N by pointing the last pointer to the node N as shown above. The next pointer of N will

point to the node N itself as there is only one node. Thus, N becomes the first as well as last node in the list.

Insert at the beginning of list:

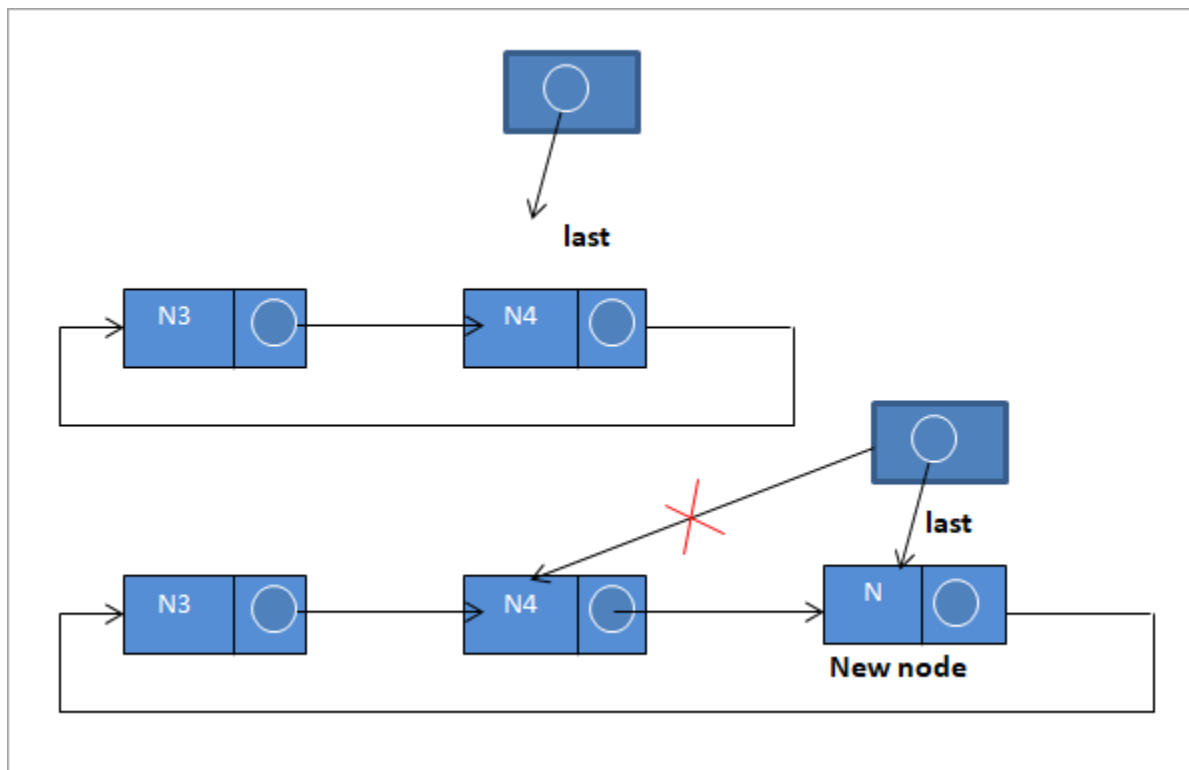


As shown in the above representation, when we add a node at the beginning of the list, the next pointer of the last node points to the new node N thereby making it a first node.

$N \rightarrow \text{next} = \text{last} \rightarrow \text{next}$

$\text{Last} \rightarrow \text{next} = N$

Insert at the end of list:



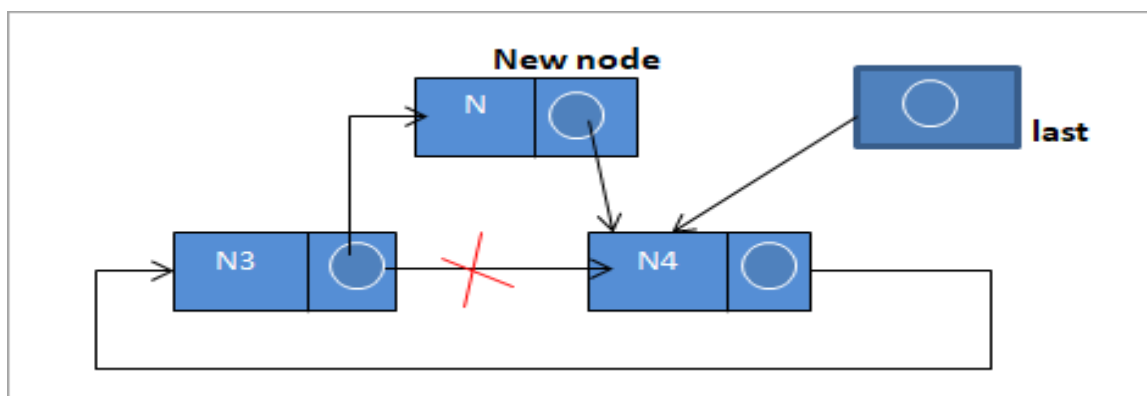
To insert a new node at the end of the list, we follow these steps:

$N \rightarrow \text{next} = \text{last} \rightarrow \text{next};$

$\text{last} \rightarrow \text{next} = N$

$\text{last} = N$

Insert in between list:



Suppose we need to insert a new node N between N3 and N4, we first need to traverse the list and locate the node after which the new node is to be inserted, in this case, its N3.

After the node is located, we perform the following steps.

```
N -> next = N3 -> next;
```

```
N3 -> next = N
```

This inserts a new node N after N3.

```
#include<iostream>
```

```
using namespace std;
```

```
struct node
```

```
{
```

```
    int value;
```

```
    node*next;
```

```
};
```

```
node*tail=NULL;
```

```
void createCircularLL()
```

```
{
```

```
    node*newNode=(struct node*)malloc(sizeof(struct node));
```

```
    cout<<"Enter the value for new node: ";
```

```
    cin>>newNode->value;
```

```
    newNode->next=NULL;
```

```
    if(tail==NULL)
```

```
    {
```

```
        tail=newNode;
```

```
        tail->next=tail;
```

```
    }
```

```
    else
```

```
    {
```

```
        newNode->next=tail->next;
```

```
        tail->next=newNode;
```

```
        tail=newNode;
```

```
    }
```

```

}

int length()
{
    int size = 0;
    node*temp = tail->next;
    do
    {
        size++;
        temp=temp->next;
    }while(temp!=tail->next);
    return size;
}

void print()
{
    node*temp = tail->next;
    do
    {
        cout<<temp->value<<" ";
        temp=temp->next;
    }while(temp!=tail->next);
    cout<<endl;
}

void insertAtHead()
{
    node* newNode=(struct node*)malloc(sizeof(struct node));

    cout<<"Enter the value for new Head node: ";
    cin>>newNode->value;
    newNode->next=NULL;
    if(tail==NULL)
    {

```

```

        tail=newNode;
        tail->next=newNode;
    }
    else
    {
        newNode->next=tail->next;
        tail->next=newNode;
    }

}

void insertAtTail()
{
    node* newNode=(struct node*)malloc(sizeof(struct node));
    cout<<"Enter the value for new Tail node: ";
    cin>>newNode->value;
    newNode->next=NULL;
    if(tail==NULL)
    {
        tail=newNode;
        tail->next=newNode;
    }
    else
    {
        newNode->next=tail->next;
        tail->next=newNode;
        tail=newNode;
    }

}

void insertAtPosition()

```

```

{
    int pos;
    int l = length();
    int i = 1 ;
    node*temp=tail->next;
    cout<<"Enter the position you wanna insert: ";
    cin>>pos;
    if(pos< 1 || pos>l)
    {
        cout<<"invalid Positon"<<endl;
        return;
    }
    if(pos==1)
    {
        insertAtHead();
        return;
    }
    while(i<pos-1)
    {
        temp=temp->next;
        i++;
    }
    node* newNode=(struct node*)malloc(sizeof(struct node));
    cout<<"Enter the value for new node: ";
    cin>>newNode->value;
    newNode->next=NULL;
    newNode->next=temp->next;
    temp->next=newNode;
}

```

```

int main()
{
    createCircularLL();
    print();
    createCircularLL();
    print();
    createCircularLL();
    print();
    insertAtHead();
    print();
    insertAtTail();
    print();
    insertAtPosition();
    print();
    cout<<"Length of LL is "<<length()<<endl;
    return 0;
}

```

Program 12: Write a C++ program for Implementation of stack using Array.

A stack is an abstract data structure that contains a collection of elements. Stack implements the LIFO mechanism i.e. the element that is pushed at the end is popped out first. Some of the principle operations in the stack are –

- Push - This adds a data value to the top of the stack.
- Pop - This removes the data value on top of the stack.
- Peek - This returns the top data value of the stack.

Implementation of stack using Code:

```

#include<iostream>

#define n 5

using namespace std;

```

```
int stack[n];

int top = -1;

void push();

void pop();

void Top();

void display();

int main()
{
    int x;

    do
    {

        cout<<"\n1: Push(): "<<endl;

        cout<<"2: Pop(): "<<endl;

        cout<<"3: Top(): "<<endl;

        cout<<"4: Display(): "<<endl;

        cout<<"0: Exit(): "<<endl;

        cin>>x;

        switch(x)
        {

            case 1:

                {

                    push();

                    break;

                }

            case 2:
```

```

        {

            pop();

            break;

        }

    case 3:

        {

            Top();

            break;

        }

    case 4:

        {

            display();

            break;

        }

    }

}while(x!=0);

return 0;

}

void push()

{

    int value;

    cout<<"Enter the value you wanna Push (Insert): ";

    cin>>value;

```

```
    if(top==n-1)
    {
        cout<<"Stack is FULL / Stack is overFlow "<<endl;
    }
    else
    {
        top++;
        stack[top]=value;
    }
}

void pop()
{
    int value;
    if(top== -1)
    {
        cout<<"Stack is Empty"<<endl;
    }
    else
    {
        value=stack[top];
        top--;
        cout<<"POP OUT value from stack is "<<value<<endl;
    }
}

void Top()
```

```
{  
  
    if(top==-1)  
  
    {  
  
        cout<<"Stack is empty there is no top "<<endl;  
  
    }  
  
    else  
  
    {  
  
        cout<<"TOP value in Stack is: "<<stack[top]<<endl;  
  
    }  
  
}  
  
void display()  
  
{  
  
    if(top==-1)  
  
    {  
  
        cout<<"Stack is empty there is no top "<<endl;  
  
    }  
  
    else  
  
    {  
  
        for(int i = top ; i >=0 ; i--)  
  
        {  
  
            cout<<stack[i]<<" ";  
  
        }  
  
    }  
  
}
```

Program 13: Write a C++ program for Implementation of stack using LinkList.

```
#include<iostream>

using namespace std;

struct node
{
    int value;
    node*next;
};

node* top = NULL;

void push()
{
    node*newNode=(struct node*)malloc(sizeof(struct node));

    cout<<"Enter the Value you wanna push in stack: ";

    cin>>newNode->value;

    newNode->next=top;

    top=newNode;
}

void pop()
{
    node* temp= top;

    if(top==NULL)
    {
        cout<<"Stack is Empty"<<endl;
    }
}
```

```

else

{

    cout<<"Delete value is "<<temp->value<<endl;

    top=top->next;

    delete temp;

}

}

void Top()

{

    if(top==NULL)

    {

        cout<<"Stack is Empty"<<endl;

        return;

    }

    cout<<"Top: "<<top->value<<endl;

}

void display()

{

    node* temp = top;

    if(top==NULL)

    {

        cout<<"Stack is empty"<<endl;

        return;

    }

    while(temp!=NULL)

```

```

        {

            cout<<temp->value<<" ";

            temp = temp->next;

        }

        cout<<endl;

    }

int main()

{

    int choice;

    do

    {

        cout<<"\n1: Push(): "<<endl;

        cout<<"2: Pop(): "<<endl;

        cout<<"3: Top(): "<<endl;

        cout<<"4: Display(): "<<endl;

        cout<<"0: Exit(): "<<endl;

        cin>>choice;

        switch(choice)

        {

            case 1:

                {

                    push();

                    break;

                }

            case 2:

```

```

        {

            pop();

            break;

        }

    case 3:

        {

            Top();

            break;

        }

    case 4:

        {

            display();

            break;

        }

    }

}while(choice!=0);

}

```

Program 14: Write a C++ program for Implementation of Queue using Array.

The queue is a basic data structure just like a stack. In contrast to stack that uses the LIFO approach, queue uses the FIFO (first in, first out) approach. With this approach, the first item that is added to the queue is the first item to be removed from the queue. Just like Stack, the queue is also a linear data structure.

In a real-world analogy, we can imagine a bus queue where the passengers wait for the bus in a queue or a line. The first passenger in the line enters the bus first as that passenger happens to be the one who had come first.

Basic operations:

The queue data structure includes the following operations:

EnQUEUE: Adds an item to the queue. Addition of an item to the queue is always done at the rear of the queue.

DEQUEUE: Removes an item from the queue. An item is removed or de-queued always from the front of the queue.

ISEMpty: Checks if the queue is empty.

isFull: Checks if the queue is full.

Peek: Gets an element at the front of the queue without removing it.

Implementation of Queue using Array:

```
#include<iostream>

using namespace std;

#define N 5

int q[N];

int front = -1;

int rear = -1;

void enqueue()

{

    int x;

    cout<<"Enter value you wanna Enque: ";

    cin>>x;

    if(rear == N-1)
```

```
{  
  
    cout<<"Que is full"<<endl;  
  
}  
  
else if(front == -1 && rear == -1)  
  
{  
  
    rear = front = 0;  
  
    q[rear]=x;  
  
}  
  
else  
  
{  
  
    rear++;  
  
    q[rear]=x;  
  
}  
  
}  
  
void deque()  
  
{  
  
    if(rear===-1 && front ===-1)  
  
    {  
  
        cout<<"Que is empty"<<endl;  
  
    }  
  
    else if (front == rear)  
  
    {  
  
        rear = front = -1;  
  
    }  
  
    else
```

```

        {

            cout<<"Deleted value: "<<q[front];

            front++;

        }

    }

void Front()

{

    if(rear== -1 && front == -1)

    {

        cout<<"Que is empty"<<endl;

    }

    else

    {

        cout<<"Current Front is: "<<q[front];

    }

}

void display()

{

    if(rear== -1 && front == -1)

    {

        cout<<"Que is empty"<<endl;

        return;

    }

    for(int i = front ; i < rear+1 ; i++)

    {

```

```

        cout<<q[i]<<" ";

    }

}

int main()

{

    int choice;

    do

    {

        cout<<"\n1: Enqueue(): "<<endl;

        cout<<"2: deque(): "<<endl;

        cout<<"3: front(): "<<endl;

        cout<<"4: Display(): "<<endl;

        cout<<"0: Exit(): "<<endl;

        cin>>choice;

        switch(choice)

        {

            case 1:

            {

                enqueue();

                break;

            }

            case 2:

            {

                deque();

```

```

        break;
    }
    case 3:
    {
        Front();
        break;
    }
    case 4:
    {
        display();
        break;
    }
}

}while(choice!=0);
}

```

Program 15: Write a C++ program for Implementation of Queue using Linked LIST.

```

#include<iostream>

using namespace std;

struct node
{
    int value;
    node* next;
};

```

```
node* front = NULL;

node* rear = NULL;

void Enqueue()

{

    node* newNode = (struct node*)malloc(sizeof(struct node));

    cout<<"Enter the value you wanna enqueue: ";

    cin>>newNode->value;

    newNode->next=NULL;

    if(front==NULL)

    {

        front = rear = newNode;

    }

    else

    {

        rear->next=newNode;

        rear=newNode;

    }

}

void deque()

{

    if(front==NULL)

    {

        cout<<"Que is empty "<<endl;

    }

    else
```

```
{  
  
    node* temp = front;  
  
    front=front->next;  
  
    delete temp;  
  
}  
  
}  
  
void TOP()  
  
{  
  
    if(front==NULL)  
  
    {  
  
        cout<<"Que is empty "<<endl;  
  
    }  
  
    else  
  
    {  
  
        cout<<"Front is "<<front->value<<endl;  
  
    }  
  
}  
  
void display()  
  
{  
  
    if(front==NULL)  
  
    {  
  
        cout<<"Que is empty "<<endl;  
  
    }  
  
    else  
  
    {
```

```

        node* temp = front ;

        while(temp!=NULL)

        {

                cout<<temp->value<<" ";

                temp=temp->next;

        }

        cout<<endl;

    }

}

int main()

{

    Enqueue();

    Enqueue();

    Enqueue();

    Enqueue();

    deque();

    deque();

    TOP();

    display();

}

```

Program 16: Write a C++ program for Implementation of Circular Queue using Array.

A queue is an abstract data structure that contains a collection of elements. Queue implements the FIFO mechanism i.e the element that is inserted first is also deleted first.

A circular queue is a type of queue in which the last position is connected to the first position to make a circle.

```
#include<iostream>

using namespace std;

#define N 5

int q[N];

int front = -1;

int rear = -1;

void enqueue()
{
    int x;

    cout<<"Enter value you wanna Enque: ";

    cin>>x;

    if(rear == N-1 && front == 0 || front == rear+1)
    {
        cout<<"Que is full"<<endl;
    }

    else if(rear==N-1)
    {
        rear=0;

        q[rear]=x;
    }

    else if(front == -1 && rear == -1)
    {
        rear = front = 0;

        q[rear]=x;
    }
}
```

```

        else

        {

            rear++;

            q[rear]=x;

        }

    }

void deque()

{

    if(rear===-1 && front ===-1)

    {

        cout<<"Que is empty"<<endl;

    }

    else if (front == rear)

    {

        rear = front = -1;

    }

    else if ( front == N-1)

    {

        cout<<"Deleted value: "<<q[front];

        front=0;

    }

    else

    {

        cout<<"Deleted value: "<<q[front];

        front++;
    }
}

```

```

    }

}

void Front()

{
    if(rear== -1 && front == -1)

    {
        cout<<"Que is empty"<<endl;

    }

    else

    {
        cout<<"Current Front is: "<<q[front];

    }

}

void display()

{
    if(rear== -1 && front == -1)

    {
        cout<<"Que is empty"<<endl;

        return;

    }

    int i = front ;

    while(i!=rear)

    {
        cout<<q[i]<<" ";

        i=(i+1)%N;
    }
}

```

```

    }

    cout<<q[rear]<<" ";

}

int main()

{

    int choice;

    do

    {

        cout<<"\n1: Enque(): "<<endl;

        cout<<"2: deque(): "<<endl;

        cout<<"3: front(): "<<endl;

        cout<<"4: Display(): "<<endl;

        cout<<"0: Exit(): "<<endl;

        cin>>choice;

        switch(choice)

        {

            case 1:

                {

                    enqueue();

                    break;

                }

            case 2:

                {

                    deque();

```

```

        break;
    }
    case 3:
    {
        Front();
        break;
    }
    case 4:
    {
        display();
        break;
    }
}

}while(choice!=0);
}

```

Program 17: Write a C++ program to create a simple tree.

- 1. Create**
- 2. Searching**
- 3. Traversal**
 - **Inorder**
 - **Preorder**
 - **Postorder**

A Binary tree is a widely used tree data structure. When each node of a tree has at most two child nodes then the tree is called a Binary tree.

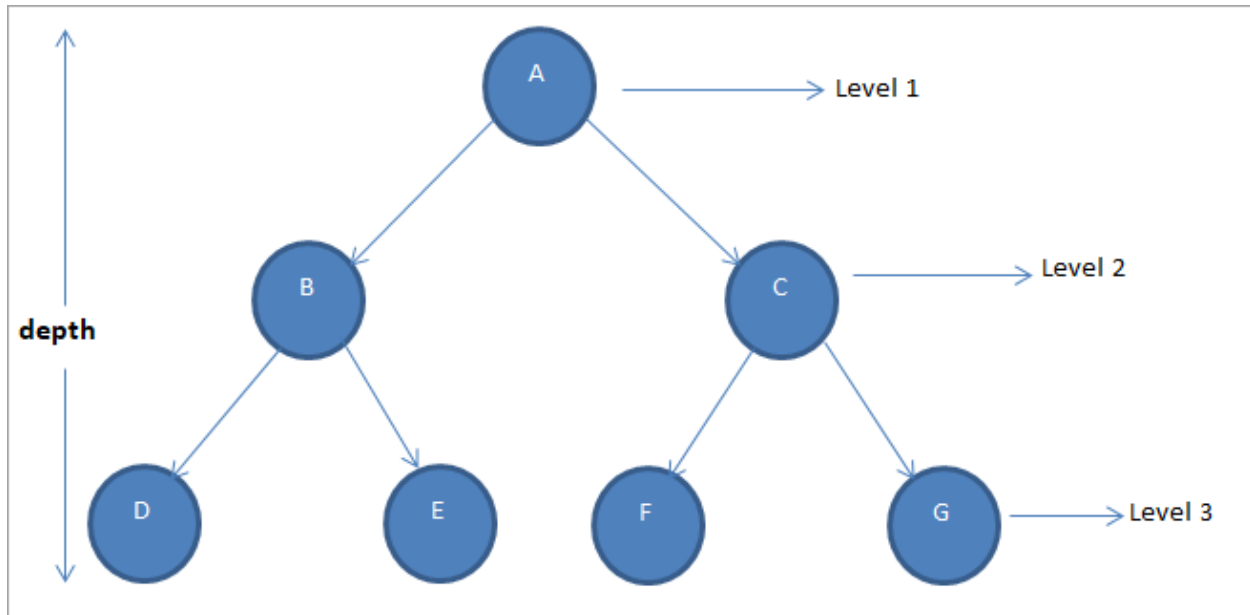
So a typical binary tree will have the following components:

A left subtree

A root node

A right subtree

A pictorial representation of a binary tree is shown below:



In a given binary tree, the maximum number of nodes at any level is 2^{l-1} where 'l' is the level number.

Thus in case of the root node at level 1, the max number of nodes = $2^{1-1} = 2^0 = 1$

As every node in a binary tree has at most two nodes, the maximum nodes at the next level will be, $2 \times 2^{l-1}$.

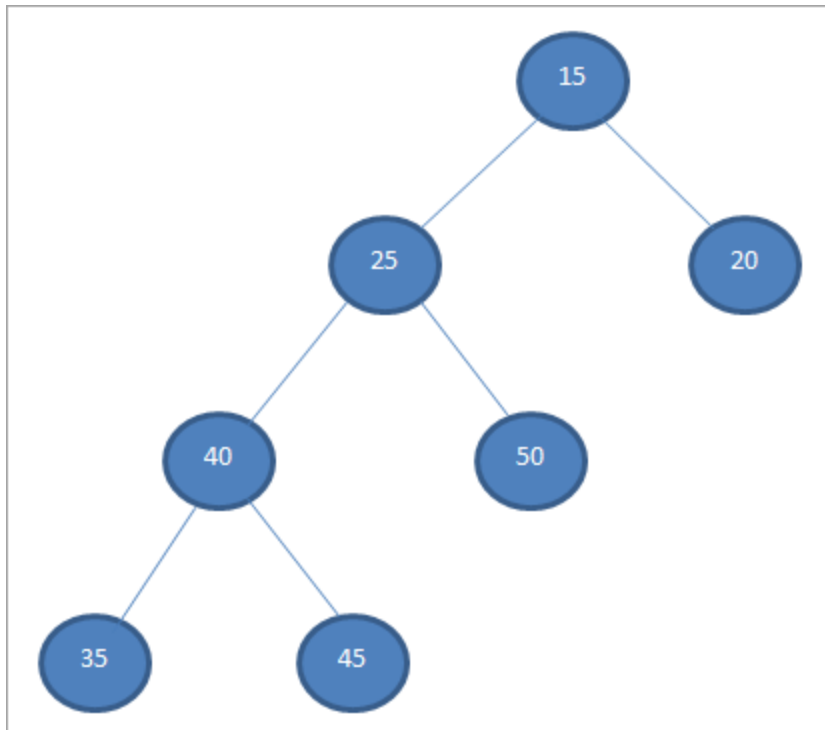
Given a binary tree of depth or height of h, the maximum number of nodes in a binary tree of height $h = 2^h - 1$.

Hence in a binary tree of height 3 (shown above), the maximum number of nodes = $2^3 - 1 = 7$.

Types of Binary Tree:

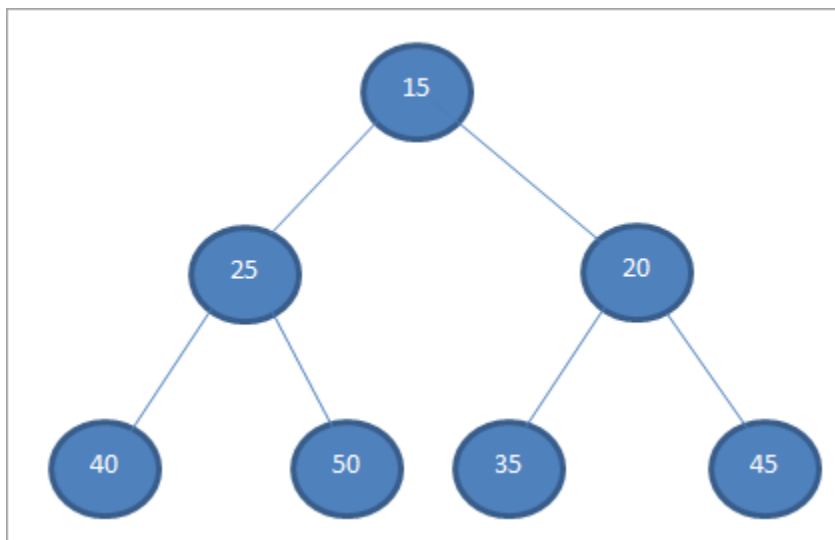
Following are the most common types of binary trees.

Full Binary Tree: A binary tree in which every node has 0 or 2 children is termed as a full binary tree.

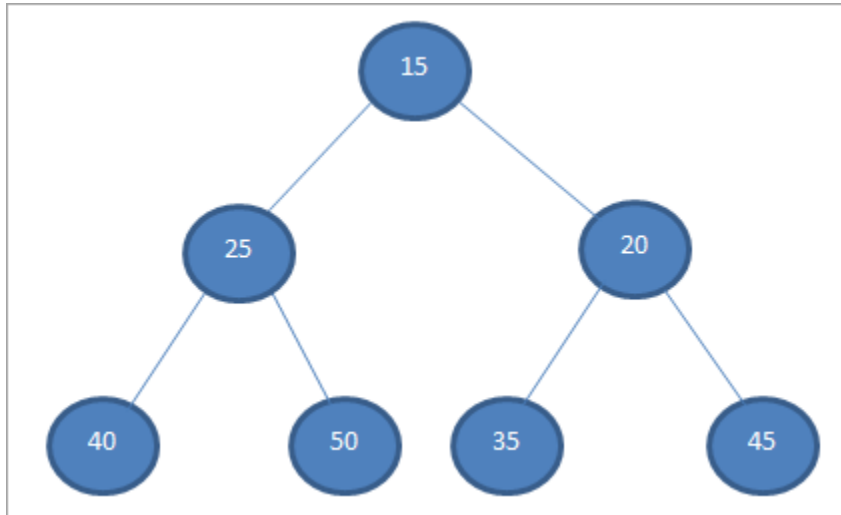


Above shown is a full binary tree in which we can see that all its nodes except the leaf nodes have two children. If L is the number of leaf nodes and ' I ' is the number of internal or non-leaf nodes, then for a full binary tree, $L = I + 1$.

Complete Binary Tree: A complete binary tree has all the levels filled except for the last level and the last level has all its nodes as much as to the left.



Strictly (perfect) Binary Tree: A binary tree is termed perfect when all its internal nodes have two children and all the leaf nodes are at the same level.



A binary tree example shown above is a perfect binary tree as each of its nodes has two children and all the leaf nodes are at the same level.

A perfect binary tree of height h has $2^h - 1$ number of nodes.

Code:

```
#include<iostream>
using namespace std;
struct node
{
    int value;
    node*left;
    node*right;
};
node* create()
{
    int x;
    cout<<"Enter the value for node (-1 for no node): ";
    cin>>x;
    node* newnode = new node();
    if(x!=-1)
```

```

    {
        return 0;
    }
    newnode->value=x;
    cout<<"Enter Left chid of "<<x<<" ";
    newnode->left=create();
    cout<<"Enter Right chid of "<<x<<" ";
    newnode->right=create();
    return newnode;
}

void preOrder(node* root)
{
    if(root==NULL)
    {
        return;
    }
    cout<<root->value<<" ";
    preOrder(root->left);
    preOrder(root->right);
}

node* search(int value,node*root)
{
    if(root==NULL)
    {
        cout<<"Value not found ";
        return;
    }
    if(root->value==value)
    {
        cout<<"Value Found ";
        return root;
    }
}

```

```

        if(value<root->value)
        {
            return search(value,root->left);
        }
        if(value>root->value)
        {
            return search(value,root->right);
        }
    }
void postOrder(node*root)
{
    if(root==NULL)
    {
        return;
    }
    postOrder(root->left);
    postOrder(root->right);
    cout<<root->value<<" ";
}
void inOrder(node* root)
{
    if(root==NULL)
    {
        return;
    }
    inOrder(root->left);
    cout<<root->value<<" ";
    inOrder(root->right);
}
int main()
{
    node* root = 0;

```

```

    root = create();

    cout<<"\n-----";
    cout<<"\nPreorder Traversal: ";
    preOrder(root);
    cout<<"\nPostOrder Traversal: ";
    postOrder(root);
    cout<<"\nInOrder Traversal: ";
    inOrder(root);
    return 0;
}

```

Program 17: Write a C++ program to create a BST.

A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties –

The left sub-tree of a node has a key less than or equal to its parent node's key.

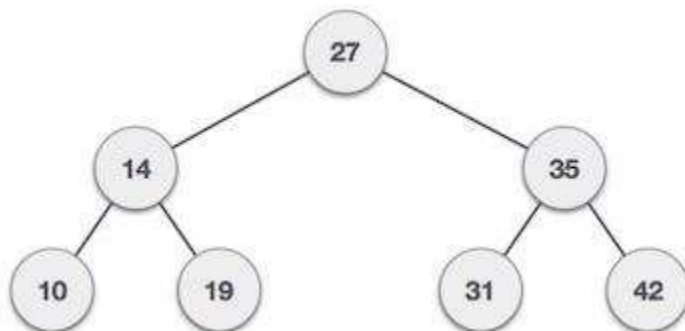
The right sub-tree of a node has a key greater than or equal to its parent node's key.

Thus, BST divides all its sub-trees into two segments; the left sub-tree and the right sub-tree and can be defined as –

Representation:

BST is a collection of nodes arranged in a way where they maintain BST properties. Each node has a key and an associated value. While searching, the desired key is compared to the keys in BST and if found, the associated value is retrieved.

Following is a pictorial representation of BST –



We observe that the root node key (27) has all less-valued keys on the left sub-tree and the higher valued keys on the right sub-tree.

Algorithm:

1. START
2. Check whether the tree is empty or not
3. If the tree is empty, search is not possible
4. Otherwise, first search the root of the tree.
5. If the key does not match with the value in the root, search its subtrees.
6. If the value of the key is less than the root value, search the left subtree
7. If the value of the key is greater than the root value, search the right subtree.
8. If the key is not found in the tree, return unsuccessful search.
9. END

Code Implementation of Insertion In BST:

```
#include<iostream>

using namespace std;

struct node
{
    int value;
    node*left;
    node*right;
};

node* Getnewnode(int value)
```

```

{

    node* newNode = new node();

    newNode->value=value;

    newNode->left=NULL;

    newNode->right=NULL;

    return newNode;

}

node* insert(int data , node* root)

{

    if(root==NULL)

    {

        root=Getnewnode(data);

        return root;

    }

    else if(data<=root->value)

    {

        root->left=insert(data,root->left);

        return root;

    }

    else

    {

        root->right = insert(data,root->right);

        return root;

    }

}

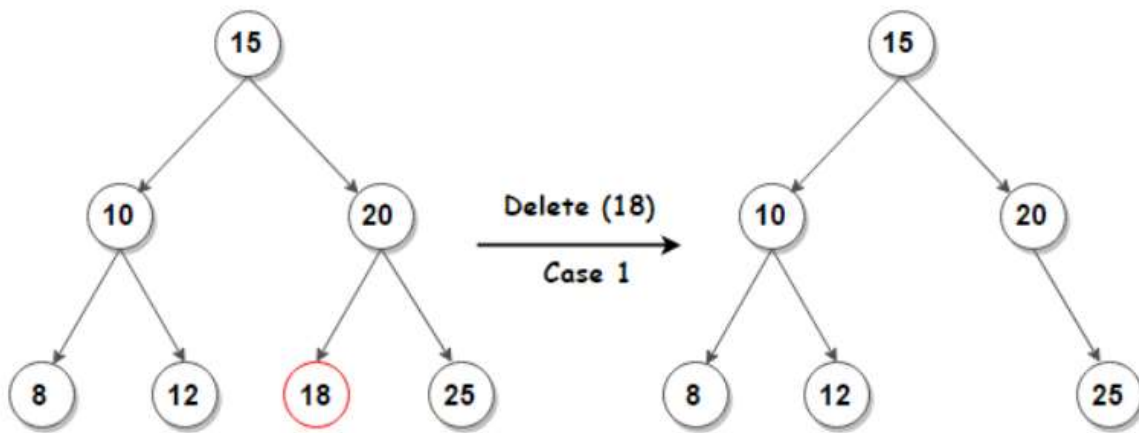
```

}

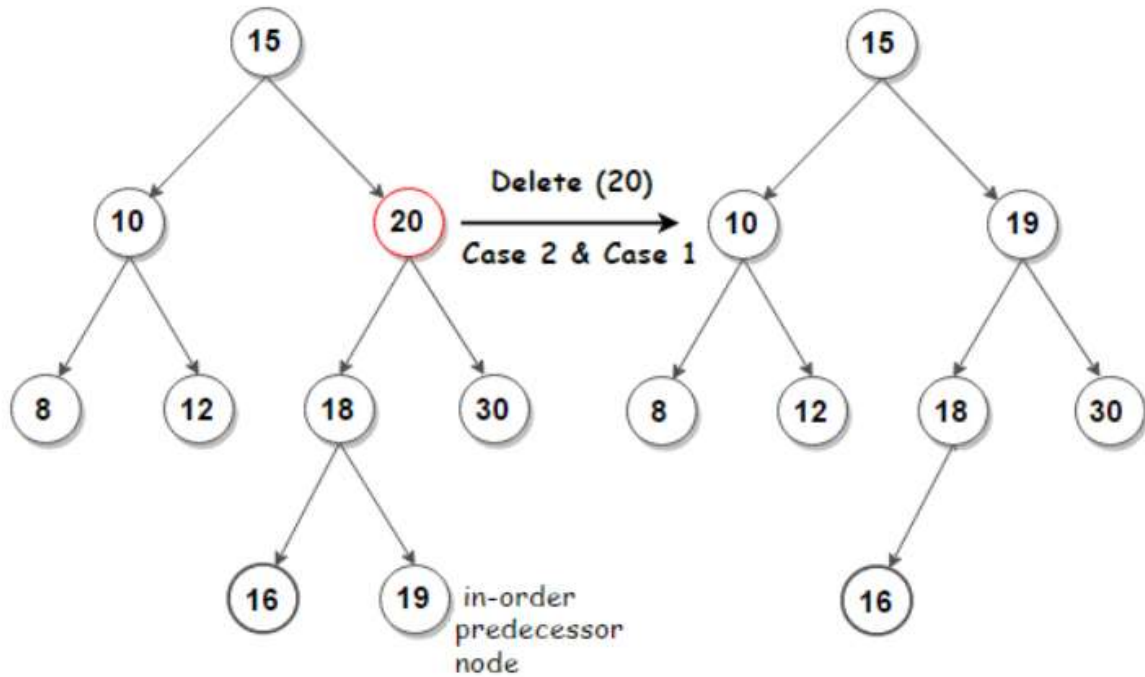
Deletion in BST:

There are three possible cases to consider deleting a node from BST:

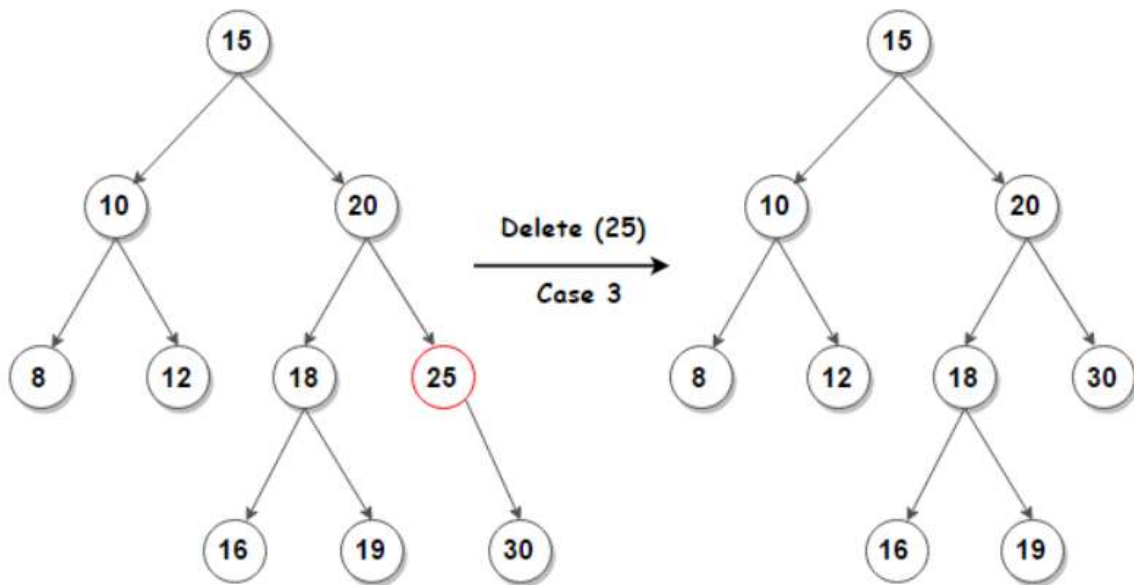
Case 1: Deleting a node with no children: remove the node from the tree.



Case 2: Deleting a node with two children: call the node to be deleted N. Do not delete N. Instead, choose either its inorder successor node or its inorder predecessor node, R. Copy the value of R to N, then recursively call delete on R until reaching one of the first two cases. If we choose the inorder successor of a node, as the right subtree is not NULL (our present case is a node with 2 children), then its inorder successor is a node with the least value in its right subtree, which will have at a maximum of 1 subtree, so deleting it would fall in one of the first 2 cases.



Case 3: Deleting a node with one child: remove the node and replace it with its child.



Broadly speaking, nodes with children are harder to delete. As with all binary trees, a node's inorder successor is its right subtree's leftmost child, and a node's inorder predecessor is the left subtree's rightmost child. In either case, this node will have zero or one child. Delete it according to one of the two simpler cases above.

Code Implementation of Deletion in BST:

```
struct node* Delete(node* root , int data)
{
    if(root==NULL)
    {
        return root;
    }
    else if(data < root->value)
    {
        root->left = Delete(root->left,data);
    }
    else if(data > root->value)
    {
        root->right = Delete(root->right,data);
    }
    else
    {
        if(root->left == NULL && root->right == NULL)
        {
            delete root;
            root = NULL;
        }
        else if (root->left == NULL)
        {
            node* temp = root;
```

```

        root = root->right;

        delete temp;
    }

    else if (root->right == NULL)
    {

        node* temp = root;

        root = root->left;

        delete temp;

    }

    else
    {

        node* temp = FindMin(root->right);

        root->value = temp->value;

        root->right = Delete(root->right,temp->value);

    }

}

return root;

}

node* FindMin(node* root)

{

    while(root->left != NULL) root = root->left;

    return root;

}

```